

RAG & LLMs

CẨM NANG VIẾT TAY — BẢN GIẢI THÍCH CHI TIẾT

16 trang cẩm nang gốc • 15 chương diễn giải • công thức, ví dụ tính tay, code



Nền tảng LLM • Transformer & Attention • Token, Embedding, Context
Prompt Engineering • Kiến trúc LLM • Pretraining, Fine-tuning, RLHF
RAG: Chunking • Embeddings • Vector DB • Retrieval • Reranking • Pipeline
Kỹ thuật RAG nâng cao • Agents • Đánh giá • Phỏng vấn

MỤC LỤC

Nội dung

Tổng quan: Bản đồ RAG & LLMs	3
1. Kiến thức nền tảng về LLM	6
2. Transformers & Attention	9
3. Tokens, Embeddings & Context Window	12
4. Prompt Engineering	15
5. Kiến trúc LLM	18
6. Pretraining & Fine-Tuning	21
7. Instruction Tuning & RLHF	24
8. Nền tảng về RAG	27
9. Document Loading & Chunking	30
10. Embeddings & Vector Databases	33
11. Retrieval & Reranking	36
12. Pipeline & Kiến trúc RAG	39
13. Các kỹ thuật RAG nâng cao	42
14. Ứng dụng LLM, Agents & Đánh giá	45
15. Phỏng vấn RAG & LLM + Kiến trúc thực tế	48
Phụ lục: Bảng thuật ngữ	51

Mỗi chương gồm **một trang ảnh gốc** của cẩm nang viết tay, tiếp theo là phần **giải thích chi tiết**: diễn giải từng ô trong ảnh, công thức kèm ví dụ tính tay, đoạn code minh họa, và các khung màu vàng **“Điểm chính / Lưu ý”** chỉ ra chỗ ảnh đơn giản hóa quá mức hoặc đã lỗi thời. Khung tím là công thức; khung xám là code Python.

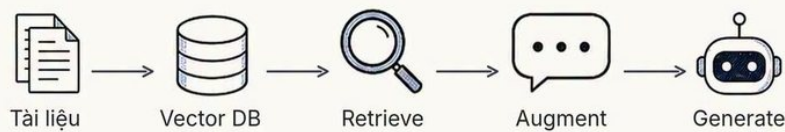
TỔNG QUAN · ẢNH GỐC

Tổng quan: Bản đồ RAG & LLMs

RAG & LLMs

- CẨM NANG VIẾT TAY -

Làm chủ RAG & LLMs từ cơ bản đến nâng cao.
Khái niệm thực tiễn. Use case thực tế.
Hỏi đáp phỏng vấn. Best practices. 😊



\\ BÊN TRONG CÓ GÌ? //

1 Pretraining & Fine-Tuning

- Pretraining là gì?
- Fine-Tuning cơ bản
- Full Fine-Tuning vs Parameter-Efficient FT
- Công cụ & Framework

2 Nền tảng về LLM

- Kiến trúc Transformer
- Tokens, Embeddings, Context
- Cơ chế Attention
- Các chiến lược Decoding

3 Instruction Tuning & RLHF

- Instruction Tuning
- Supervised Fine-Tuning (SFT)
- Reward Modeling
- Quy trình RLHF

4 Nền tảng về RAG

- RAG là gì?
- RAG vs Fine-Tuning
- Khi nào nên dùng RAG?
- Lợi ích & hạn chế

5 Document Loading & Chunking

- Document Loaders
- Loại file & nguồn dữ liệu
- Chiến lược Text Splitting
- Chunk Size & Overlap

6 Embeddings & Vector DBs

- Embeddings là gì?
- Các Embedding Model
- Vector Databases
- Similarity Search

7 Retrieval & Reranking

- Các chiến lược Retrieval
- Top-K, MMR, Hybrid Search
- Các Reranking Model
- Cross-Encoder Reranking

8 Pipeline & kiến trúc RAG

- Luồng RAG End-to-End
- Thành phần & luồng dữ liệu
- Naive RAG vs Advanced RAG
- Các mẫu kiến trúc

9 Kỹ thuật RAG nâng cao

- Query Transformation
- Multi-Query Retrieval
- HyDE, Self-RAG, FLARE
- Graph RAG, Agentic RAG

10 Ứng dụng của LLM

- Chatbot & trợ lý ảo
- Hệ thống Search & Q&A
- Tóm tắt văn bản
- Tạo nội dung

11 Agents & Tools

- AI Agent là gì?
- Kiến trúc Agent
- Tools & Function Calling
- Planning & Memory

12 Đánh giá & Metrics

- Metrics đánh giá RAG
- Metrics đánh giá LLM
- Đánh giá bởi con người
- Tracing & Observability

13 Use case thực tế

- Use case doanh nghiệp
- Ứng dụng trong các ngành
- Case study

14 Hỏi đáp phỏng vấn RAG & LLM

- Câu hỏi khái niệm
- Câu hỏi System Design
- Q&A theo tình huống

15 Best practices & lỗi thường gặp

- Best practices
- Lỗi thường gặp
- Mẹo tối ưu

TỔNG QUAN · GIẢI THÍCH CHI TIẾT**Tổng quan: Bản đồ RAG & LLMs****Bức tranh lớn trước khi đi vào chi tiết**

Trang mở đầu của bộ cẩm nang tóm tắt toàn bộ hành trình bằng một chuỗi năm bước: **Tài liệu** → **Vector DB** → **Retrieve** → **Augment** → **Generate**. Đây chính là "xương sống" của một hệ thống RAG (Retrieval-Augmented Generation). Hiểu được chuỗi này là hiểu được 80% lý do RAG tồn tại.

- **Tài liệu**: kiến thức riêng của bạn (quy chế, sổ tay, hợp đồng, tài liệu kỹ thuật...). LLM không được huấn luyện trên những tài liệu này nên không biết chúng.
- **Vector DB**: tài liệu được cắt nhỏ (chunk), mỗi chunk được biến thành một vector số (embedding) và lưu vào cơ sở dữ liệu chuyên tìm kiếm vector.
- **Retrieve**: khi người dùng hỏi, câu hỏi cũng được biến thành vector; hệ thống tìm những chunk có vector "gần" nhất.
- **Augment**: các chunk tìm được được chèn vào prompt cùng câu hỏi, tạo thành "prompt tăng cường".
- **Generate**: LLM đọc prompt tăng cường và sinh câu trả lời dựa trên ngữ cảnh đã cho, thay vì chỉ dựa vào trí nhớ của nó.

Năm lợi ích ghi trên dải băng

Lợi ích	Ý nghĩa thực tế
Truy xuất thông minh hơn	Tìm theo ý nghĩa (semantic), không chỉ khớp từ khóa.
Bổ sung ngữ cảnh tốt hơn	LLM nhận đúng đoạn tài liệu liên quan thay vì phải "đoán".
Sinh câu trả lời chính xác	Câu trả lời bám vào nguồn, giảm bịa đặt (hallucination).
Mở rộng linh hoạt	Thêm tài liệu mới chỉ cần index lại, không cần huấn luyện lại model.
Xây dựng niềm tin	Có thể trích dẫn nguồn, người dùng kiểm chứng được.

Cách đọc cuốn sách này

Bộ ảnh gồm 15 chủ đề, đi từ nền móng (LLM là gì, Transformer, token) lên kỹ thuật huấn luyện (pretraining, fine-tuning, RLHF), rồi sang RAG (chunking, embedding, retrieval, reranking, pipeline, kỹ thuật nâng cao), cuối cùng là ứng dụng, agent, đánh giá và câu hỏi phỏng vấn. Mỗi chương trong sách này gồm hai phần:

- **Ảnh gốc** — giữ nguyên để bạn ôn nhanh bằng hình.
- **Giải thích cận kề** — diễn giải từng ô trong ảnh, bổ sung công thức, ví dụ tính tay, đoạn code minh họa, và **các điểm cần nhấn mạnh** khi ảnh đơn giản hóa quá mức hoặc đã lỗi thời.

Lộ trình gợi ý: Chương 1-5 (nền tảng LLM) → Chương 8-12 (RAG cốt lõi, nên vừa đọc vừa code) → Chương 13 (nâng cao) → Chương 14-15 (agent, đánh giá, ôn phỏng vấn). Chương 6-7 (huấn luyện) có thể đọc sau nếu mục tiêu trước mắt là xây chatbot RAG.

Ví dụ xuyên suốt

Để các khái niệm không bị rời rạc, nhiều chương sẽ dùng chung một ví dụ: **chatbot hỏi đáp quy chế đào tạo của một trường đại học**. Kho tài liệu là các văn bản quy chế (PDF/DOCX) chia theo Chương - Điều - Khoản; người dùng là sinh viên hỏi những câu như "Điểm trung bình bao nhiêu thì bị cảnh báo học tập?" hay "Một học kỳ được đăng ký tối đa bao nhiêu tín chỉ?". Đây là một bài toán RAG kinh điển: dữ liệu riêng, cần chính xác tuyệt đối, cần trích dẫn Điều/Khoản.

CHƯƠNG 1 · ẢNH GỐC

1. Kiến thức nền tảng về LLM



1. KIẾN THỨC NỀN TẢNG VỀ LLM

RAG & LLMs
SERIES VIẾT TAY
v1.0

• HIỂU NỀN TẢNG. XÂY DỰNG TƯƠNG LAI. •

LLM là gì?

LLM (Large Language Model) là một mô hình AI được huấn luyện trên lượng dữ liệu văn bản khổng lồ để hiểu, tạo ra và tương tác bằng ngôn ngữ của con người.

- ✓ Học các pattern từ văn bản
- ✓ Hiểu ngữ cảnh
- ✓ Tạo phản hồi giống con người
- ✓ Có thể fine-tune cho các tác vụ cụ thể

Ví dụ về các LLM

GPT-4o
OpenAIGemini 1.5
GoogleClaude 3
AnthropicLlama 3
MetaMistral Large
Mistral AI

Vì sao LLM quan trọng?

- ⚡ Vận hành chatbot, copilot và trợ lý ảo
- ⚡ Tự động hóa việc tạo nội dung và tóm tắt
- ⚡ Hiểu và phân tích văn bản phi cấu trúc
- ⚡ Cho phép tương tác tự nhiên với dữ liệu và hệ thống
- ⚡ Nền tảng cho nhiều ứng dụng AI (RAG, Agents, v.v.)

LLM hoạt động thế nào? (Tổng quan)



Năng lực chính

Sinh văn bản

Tạo câu trả lời, câu chuyện, code, email, v.v.

Thấu hiểu

Tóm tắt, phân loại, dịch thuật, trích xuất thông tin.

Suy luận

Giải quyết vấn đề, lập luận logic, trả lời các câu hỏi phức tạp.

Làm theo chỉ dẫn

Tuân theo hướng dẫn và các ràng buộc của người dùng.

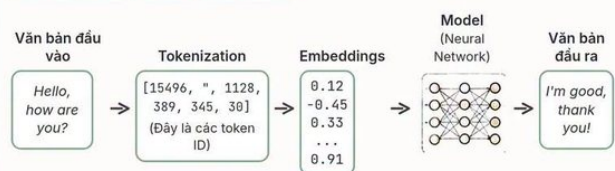
Thích ứng

Thích ứng với tác vụ mới thông qua fine-tuning hoặc prompting.

Khái niệm cốt lõi

- 1 **Model**: Mạng neural đã được huấn luyện (ví dụ: GPT-4o).
- 2 **Parameters**: Các trọng số (weights) mà model học được (hàng tỷ trong các LLM hiện đại).
- 3 **Token**: Đơn vị nhỏ nhất của văn bản (từ/mảnh từ/ký tự).
- 4 **Context Window**: Lượng văn bản tối đa mà model có thể "ghi nhớ" cùng lúc.
- 5 **Inference**: Dùng model để sinh output từ input.
- 6 **Probability**: LLM dự đoán token tiếp theo dựa trên xác suất.

Văn bản được xử lý thế nào?



💡 LLM không hiểu từ ngữ như con người. Chúng làm việc với các con số (token) và pattern.

Điểm mạnh

- ✓ Hiểu ngôn ngữ tự nhiên
- ✓ Tạo văn bản giống con người
- ✓ Tổng quát hóa cho nhiều tác vụ
- ✓ Giảm thời gian phát triển
- ✓ Tăng năng suất

Hạn chế

- ✗ Có thể tạo ra thông tin sai (hallucinations)
- ✗ Bị giới hạn bởi dữ liệu huấn luyện
- ✗ Không có kiến thức real-time (trừ khi được kết nối)
- ✗ Tốn nhiều tài nguyên tính toán
- ✗ Vấn đề đạo đức & quyền riêng tư

Các loại LLM (theo mục đích sử dụng)

- ✦ **Base Models**: Huấn luyện trên dữ liệu khổng lồ, dùng cho mục đích chung (ví dụ: Llama, GPT base)
- ✦ **Instruction Tuned**: Huấn luyện trên dữ liệu làm theo chỉ dẫn (ví dụ: Alpaca, Vicuna)
- ✦ **Chat Models**: Tối ưu cho hội thoại (ví dụ: ChatGPT, Claude)
- ✦ **Code Models**: Chuyên về sinh code (ví dụ: CodeLlama, StarCoder)
- ✦ **Multimodal Models**: Làm việc với văn bản, hình ảnh, âm thanh, video

CHƯƠNG 1 · GIẢI THÍCH CHI TIẾT

Kiến thức nền tảng về LLM

1. LLM là gì — hiểu cho đúng

LLM (Large Language Model) là một mạng neural rất lớn được huấn luyện với một nhiệm vụ duy nhất nghe có vẻ đơn giản: **dự đoán token tiếp theo**. Cho chuỗi "Thủ đô của Việt Nam là", model tính xác suất cho mọi token trong từ vựng và token "Hà" có xác suất cao nhất. Lặp lại việc này hàng nghìn tỷ lần trên dữ liệu khổng lồ, model buộc phải "nén" được ngữ pháp, kiến thức thế giới, và cả một phần khả năng suy luận vào các tham số của nó.

Bốn dấu tick trong ảnh nên được hiểu như sau:

- **Học các pattern từ văn bản**: không có "cơ sở dữ liệu kiến thức" bên trong; kiến thức nằm phân tán trong hàng tỷ trọng số.
- **Hiểu ngữ cảnh**: nhờ cơ chế attention, nghĩa của một từ được xác định bởi các từ xung quanh ("bank" trong "river bank" khác "bank account").
- **Tạo phản hồi giống con người**: vì được học từ văn bản người viết, và được tinh chỉnh thêm bằng instruction tuning/RLHF (Chương 7).
- **Có thể fine-tune**: tiếp tục huấn luyện trên dữ liệu chuyên biệt (Chương 6).

2. LLM hoạt động thế nào (tổng quan 4 bước)

Sơ đồ "Dữ liệu văn bản khổng lồ → Training → Model học các pattern → Sinh văn bản" tách bạch hai giai đoạn rất khác nhau về chi phí:

Giai đoạn	Diễn ra khi nào	Chi phí
Training (huấn luyện)	Một lần, do nhà phát triển model thực hiện	Hàng nghìn GPU, hàng tuần-tháng
Inference (suy luận)	Mỗi lần người dùng gửi câu hỏi	Một GPU/vài GPU, mili-giây đến giây

Khi bạn dùng LLM qua API hay chạy bằng Ollama, bạn chỉ làm inference. RAG cũng hoạt động hoàn toàn ở tầng inference — đó là lý do RAG rẻ hơn fine-tuning rất nhiều.

3. Sáu khái niệm cốt lõi

- **Model**: tập hợp kiến trúc (cách các lớp nối với nhau) + trọng số đã học.
- **Parameters**: số trọng số. Model "7B" có khoảng 7 tỷ tham số; lưu ở dạng 16-bit cần khoảng $7 \times 2 = 14$ GB bộ nhớ, ở dạng lượng tử hóa 4-bit cần khoảng 3,5–4,5 GB. Đây là phép tính bạn nên làm nhằm khi chọn model cho GPU của mình.
- **Token**: đơn vị văn bản mà model xử lý (Chương 3).
- **Context window**: số token tối đa model xử lý trong một lần gọi, gồm cả đầu vào và đầu ra.
- **Inference**: chạy model để sinh output.
- **Probability**: output của model ở mỗi bước là một *phân phối xác suất* trên toàn bộ từ vựng; cách ta chọn token từ phân phối đó gọi là decoding (temperature, top-k, top-p — xem Chương 15).

4. Văn bản được xử lý thế nào

Chuỗi "Văn bản → Tokenization → Embeddings → Neural Network → Văn bản đầu ra" là luồng dữ liệu của một lần sinh. Câu ghi chú trong ảnh rất quan trọng: **LLM không đọc chữ, nó làm việc với các con số**. "Hello, how are you?" trở thành dãy ID như [15496, 11, 703, 389, 345, 30]; mỗi ID tra ra một vector; các vector đi qua nhiều lớp Transformer; lớp cuối cho ra xác suất token tiếp theo.

5. Các loại LLM theo mục đích

Loại	Đặc điểm	Khi nào dùng
Base model	Chỉ biết nối tiếp văn bản, không "nghe lời"	Làm nền để fine-tune
Instruction-tuned	Được dạy làm theo chỉ dẫn	Tác vụ một lượt: tóm tắt, phân loại
Chat model	Tối ưu hội thoại nhiều lượt, có RLHF	Chatbot, trợ lý
Code model	Huấn luyện thêm trên mã nguồn	Sinh/sửa code
Multimodal	Nhận ảnh, âm thanh, video	Đọc tài liệu scan, phân tích hình

Thử nghiệm nhỏ để thấy khác biệt: đưa prompt "Thủ đô của Pháp là gì?" cho một base model, nó có thể trả lời bằng... một loạt câu hỏi tương tự ("Thủ đô của Đức là gì? Thủ đô của Ý là gì?") vì nó chỉ đang nối tiếp một văn bản trông như danh sách câu hỏi. Chat model thì trả lời "Paris".

6. Điểm mạnh và hạn chế

Hạn chế quan trọng nhất cho người làm RAG là **hallucination**: model sinh ra văn bản *trôi chảy* nhưng *sai*, vì mục tiêu huấn luyện là sinh token có xác suất cao chứ không phải sinh sự thật. Hai hạn chế tiếp theo — "giới hạn bởi dữ liệu huấn luyện" và "không có kiến thức real-time" — chính là hai lý do ra đời của RAG.

Đính chính / cập nhật: Ảnh dùng các model thế hệ 2024 (GPT-4o, Gemini 1.5, Claude 3, Llama 3, Mistral Large). Lĩnh vực này đổi rất nhanh; tên model chỉ mang tính minh họa — hãy tra trang tài liệu chính thức của từng nhà cung cấp để biết model và context window hiện hành. Với dự án tự host, các họ model mở (Llama, Qwen, Mistral, Gemma...) là lựa chọn phổ biến.

CHƯƠNG 2 · ẢNH GỐC

2. Transformers & Attention

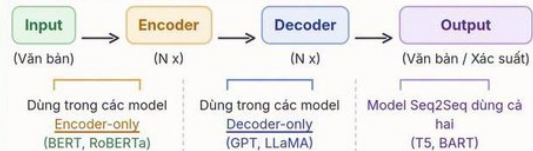
2. TRANSFORMERS
&
ATTENTIONRAG & LLMs
SERIES VIẾT TAY
v1.0

TRÁI TIM CỦA CÁC LLM HIỆN ĐẠI ♥

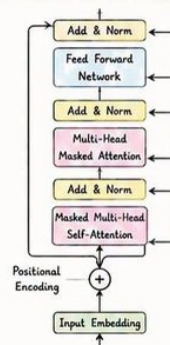
Transformer là gì?

Transformer là một kiến trúc deep learning được giới thiệu trong bài báo "Attention Is All You Need" (2017). Nó chỉ dựa vào cơ chế attention, loại bỏ recurrence và convolution.

- ✓ Xử lý toàn bộ chuỗi song song
- ✓ Nắm bắt được các phụ thuộc xa (long-range dependencies)
- ✓ Khả năng mở rộng cao và hiệu quả
- ✓ Nền tảng của các model như GPT, BERT, T5, PaLM, v.v.

Tổng quan kiến trúc Transformer**Các thành phần chính**

- 1 **Input Embedding**: Chuyển token thành các vector dày đặc (dense vector).
- 2 **Positional Encoding**: Thêm thông tin vị trí (vì transformer không có recurrence).
- 3 **Multi-Head Attention**: Cơ chế cốt lõi để tập trung vào những phần quan trọng của chuỗi.
- 4 **Add & Norm**: Residual connection + Layer Normalization.
- 5 **Feed Forward Network**: Hai lớp linear kèm hàm kích hoạt (activation).
- 6 **Masking**: Ngăn việc nhìn thấy các token tương lai (trong decoder).

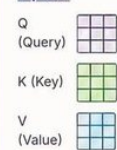
Transformer Block (Decoder)

- ⚡ **Masked Multi-Head Self-Attention**: Mỗi vị trí chỉ chú ý đến các vị trí phía trước.
- ⚡ **Multi-Head Masked Attention**: Chú ý lên output của encoder (trong các model encoder-decoder).
- ⚡ **Feed Forward Network**: Áp dụng theo từng vị trí và giống hệt nhau.
- ⚡ **Add & Norm**: Residual connection + LayerNorm giúp ổn định và cho gradient tốt hơn.

"Attention giúp model tập trung vào điều quan trọng nhất."

Self-Attention (Ý tưởng cốt lõi)

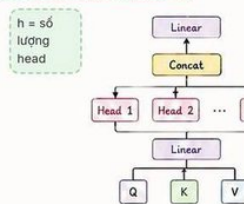
Mỗi từ nhìn vào tất cả các từ trong chuỗi và quyết định mức độ chú ý dành cho từng từ.

**Scaled Dot-Product Attention****Inputs:****Công thức:**

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Trong đó:

d_k = số chiều của key
 QK^T = điểm tương đồng (similarity score)
 Softmax = chuyển điểm số thành xác suất

Multi-Head Attention

- ⚡ Chạy nhiều phép attention song song.
- ⚡ Mỗi head học một cách biểu diễn khác nhau.
- ⚡ Giúp model nắm bắt các mối quan hệ đa dạng.

Thường h = 8, 12, 16...

Positional Encoding

Vì transformer không có recurrence, ta thêm thông tin vị trí vào embeddings.

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

→ Cộng vào input embeddings

Điều này giúp model hiểu thứ tự của các token. ★

Các loại Transformer Model

- ★ **Encoder-Only**: BERT, RoBERTa, DeBERTa (tác vụ hiểu - Understanding)
- ★ **Decoder-Only**: GPT, LLaMA, Mistral (tác vụ sinh - Generation)
- ★ **Encoder-Decoder**: T5, BART, mT5 (tác vụ Seq2Seq)

Vì sao là Transformers?

- ✓ Nắm bắt phụ thuộc xa
- ✓ Song song hóa cao
- ✓ Hiệu năng tốt hơn trên dữ liệu lớn
- ✓ Mở rộng tới hàng tỷ tham số
- ✓ Nền tảng của các LLM hiện đại



CHƯƠNG 2 · GIẢI THÍCH CHI TIẾT

Transformers & Attention

1. Vì sao Transformer thay thế RNN

Trước 2017, xử lý chuỗi chủ yếu dùng RNN/LSTM: đọc từng token một, trạng thái ẩn truyền từ bước này sang bước sau. Hai vấn đề: (1) không song song hóa được vì bước t phải chờ bước $t-1$; (2) thông tin từ rất xa bị "phai" dần. Bài báo *Attention Is All You Need* (Vaswani và cộng sự, 2017) bỏ hẳn recurrence, cho mọi token nhìn trực tiếp vào mọi token khác trong một phép nhân ma trận. Kết quả: huấn luyện song song trên GPU cực hiệu quả và nắm bắt phụ thuộc xa tốt — điều kiện cần để mở rộng lên hàng trăm tỷ tham số.

2. Ba họ kiến trúc

Họ	Attention	Ví dụ	Thế mạnh
Encoder-only	Hai chiều (nhìn cả trái lẫn phải)	BERT, RoBERTa	Hiểu: phân loại, embedding, reranker
Decoder-only	Một chiều (chỉ nhìn bên trái)	GPT, LLaMA, Mistral	Sinh văn bản — hầu hết LLM ngày nay
Encoder-decoder	Encoder hai chiều + decoder có cross-attention	T5, BART	Dịch, tóm tắt (seq2seq)

Liên hệ với RAG: **embedding model và cross-encoder reranker thường là encoder-only** (họ BERT), còn LLM sinh câu trả lời là decoder-only. Một hệ RAG thường dùng cả hai họ.

3. Self-attention: tính tay từng bước

Mỗi token có vector đầu vào x . Ta nhân với ba ma trận học được để có ba vai trò:

$$Q = X \cdot W_Q \quad K = X \cdot W_K \quad V = X \cdot W_V$$

- **Query (Q):** "tôi đang tìm thông tin gì?"
- **Key (K):** "tôi chứa loại thông tin gì?" (để người khác so khớp)
- **Value (V):** "nội dung tôi sẽ đóng góp nếu được chú ý"

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

Ví dụ số nhỏ với 3 token, $d_k = 2$. Giả sử query của token 3 là $q = [1, 0]$, các key là $k_1 = [1, 0]$, $k_2 = [0, 1]$, $k_3 = [1, 1]$.

- Điểm thô $q \cdot k$: 1, 0, 1. Chia cho $\sqrt{2} \approx 1,414$: 0,707; 0; 0,707.
- Softmax: $e^{0,707} \approx 2,03$; $e^0 = 1 \rightarrow$ tổng 5,06 $\rightarrow$ trọng số $\approx 0,40$; 0,20; 0,40.
- Đầu ra = $0,40 \cdot v_1 + 0,20 \cdot v_2 + 0,40 \cdot v_3$: token 3 "lấy" thông tin chủ yếu từ token 1 và chính nó.

Vì sao chia $\sqrt{d_k}$? Khi d_k lớn (ví dụ 128), tích vô hướng của hai vector ngẫu nhiên có phương sai tỉ lệ với d_k , giá trị rất lớn làm softmax gần như one-hot, gradient gần bằng 0. Chia $\sqrt{d_k}$ đưa phương sai về khoảng 1, giúp huấn luyện ổn định.

4. Multi-head attention

Thay vì một phép attention với d_{model} chiều, ta chia thành h "đầu", mỗi đầu d_{model}/h chiều, chạy song song rồi nối (concat) lại và chiếu qua một lớp linear. Mỗi đầu có thể chuyên một loại quan hệ: đầu này theo dõi quan hệ chủ-vị, đầu kia theo dõi đại từ trở về danh từ nào. Với $d_{\text{model}} = 4096$ và $h = 32$, mỗi đầu có 128 chiều.

5. Masking (causal mask)

Trong decoder, khi huấn luyện ta đưa cả câu vào cùng lúc nhưng token ở vị trí i **không được nhìn** token $i+1, i+2...$ (nếu không, model "nhìn trộm đáp án"). Kỹ thuật: cộng $-\infty$ vào các ô phía trên đường chéo của ma trận $Q \cdot K^T$ trước softmax, khiến trọng số tại đó bằng 0.

6. Positional encoding

Attention bản chất là phép toán trên *tập hợp* — nếu đảo thứ tự token, kết quả chỉ bị đảo theo. Do đó phải "tiêm" thông tin vị trí. Công thức sin/cos trong ảnh là của bài báo gốc:

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{2i/d}) \quad \text{PE}(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{2i/d})$$

Mỗi cặp chiều dao động với một tần số khác nhau, giống các kim đồng hồ quay nhanh chậm khác nhau, tạo ra một "dấu vân tay" duy nhất cho mỗi vị trí.

7. Add & Norm, Feed-Forward

- **Residual connection** ($x + \text{Sublayer}(x)$): cho gradient một "đường cao tốc" đi thẳng qua nhiều lớp, giúp huấn luyện mạng sâu hàng chục lớp.
- **LayerNorm**: chuẩn hóa vector về trung bình 0, phương sai 1 trên từng token.
- **FFN**: hai lớp linear, thường mở rộng 4 lần ($4096 \rightarrow 16384 \rightarrow 4096$) với hàm kích hoạt phi tuyến ở giữa. Phần lớn tham số của LLM nằm ở FFN; nhiều nghiên cứu cho thấy đây là nơi "lưu" phần lớn kiến thức dạng sự kiện.

Điểm chính: (1) Trong ô "Transformer Block (Decoder)", dòng "Multi-Head Masked Attention: chú ý lên output của encoder" thực chất là **cross-attention** (encoder-decoder attention) — lớp này **không** dùng causal mask. Sơ đồ ba lớp con là decoder của Transformer gốc; decoder của GPT/LLaMA chỉ có hai lớp con (masked self-attention + FFN). (2) Bài báo gốc dùng Post-LN (chuẩn hóa sau khi cộng), còn LLM hiện đại dùng Pre-LN/RMSNorm (chuẩn hóa trước) vì ổn định hơn. (3) Positional encoding sin/cos hầu như đã được thay bằng **RoPE** (Rotary Position Embedding) trong LLaMA, Qwen, Mistral...

CHƯƠNG 3 · ẢNH GỐC

3. Tokens, Embeddings & Context Window



3. TOKENS, EMBEDDINGS & CONTEXT WINDOW



Biến văn bản thành ý nghĩa cho LLM ❤️

1. TOKENS

Token là đơn vị cơ bản của văn bản mà LLM hiểu được. Chúng có thể là từ, subword, hoặc thậm chí là ký tự.

Ví dụ:

Văn bản: I love building AI products! 😊

Word-level Tokens:

I love building AI products ! 😊

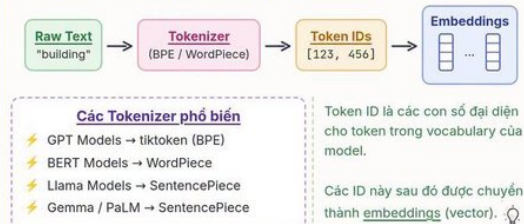
Subword Tokens (ví dụ: BPE):

I love build ing AI product s ! 😊

Điểm chính:

- LLM dùng subword tokenization (như BPE / WordPiece).
- Chuyển mọi văn bản thành một chuỗi token ID.
- Vocabulary nhỏ hơn, xử lý tốt hơn các từ hiếm & lỗi chính tả.

Tokenization hoạt động thế nào?



2. EMBEDDINGS

Embeddings là các biểu diễn vector dày đặc (dense vector) của token. Chúng nắm bắt ý nghĩa và mối quan hệ giữa các từ.

Ví dụ:

Token	Embedding Vector (Dimension = 5)	Ý nghĩa
king	[0.21, -0.13, 0.45, 0.33, -0.52]	Hoàng gia, Nam
queen	[0.20, -0.11, 0.44, 0.31, -0.49]	Hoàng gia, Nữ
man	[0.19, -0.30, 0.10, 0.25, -0.10]	Nam
woman	[0.18, -0.28, 0.11, 0.24, -0.12]	Nữ

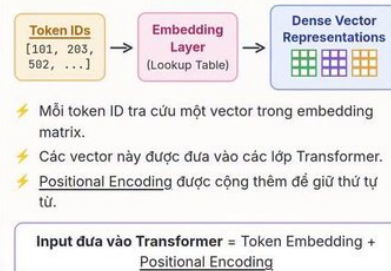


Nghĩa tương tự thì nằm gần nhau hơn trong không gian vector.

Điểm chính:

- Embeddings ánh xạ token sang vector nhiều chiều.
- Các token tương đồng về ngữ nghĩa có vector tương tự.
- Model học các embedding này trong quá trình training.
- Các embedding model phổ biến: word2vec, GloVe, FastText, BERT, OpenAI Ada.

Embeddings được dùng trong LLM thế nào



3. CONTEXT WINDOW

Context Window là số lượng token tối đa mà LLM có thể xử lý trong một lần. Nó bao gồm cả token đầu vào (prompt) và đầu ra (response).

Ví dụ:



Điểm chính:

- Khi context window đầy, các token cũ hơn sẽ bị quên.
- Context window lớn hơn = hiểu tài liệu dài tốt hơn.
- Mỗi model có kích thước context khác nhau:

Model	Context Window
GPT-4 Turbo	128K token
Claude 3 Opus	200K token
Llama 3 (8B/70B)	8K - 128K token
Gemini 1.5 Pro	1M token

Vì sao quan trọng?

- Context dài hơn = tốt hơn cho RAG, tóm tắt, code, hội thoại.
- Giúp model ghi nhớ nhiều thông tin hơn.

Nhiều context hơn, thông minh hơn! ❤️

Context Window trong thực tế (Ví dụ)

Bạn đặt câu hỏi về một tài liệu 50 trang.

30 trang đầu (Trong window) ✓	20 trang cuối (Ngoài window) ✗
-------------------------------	--------------------------------

LLM chỉ "nhìn thấy" được 30 trang đầu tại một thời điểm.

Giải pháp:

- Chia nhỏ tài liệu (chunking).
- Dùng RAG để truy xuất các chunk liên quan.
- Tóm tắt và chỉ đưa vào thông tin thiết yếu.



CHƯƠNG 3 · GIẢI THÍCH CHI TIẾT

Tokens, Embeddings & Context Window

1. Tokenization: vì sao không dùng nguyên từ?

Có ba cách cắt văn bản: theo ký tự (từ vựng nhỏ nhưng chuỗi rất dài), theo từ (chuỗi ngắn nhưng từ vựng khổng lồ, gặp từ mới là "bó tay"), và theo **subword** — điểm cân bằng. "building" có thể là [build, ing]; một từ hiếm như "RAGification" vẫn cắt được thành các mảnh đã biết. Kích thước từ vựng của LLM hiện đại thường từ 32.000 đến khoảng 200.000 token.

BPE (Byte-Pair Encoding) hoạt động như sau: bắt đầu từ các ký tự (hoặc byte), đếm cặp liên kế xuất hiện nhiều nhất trong kho dữ liệu, gộp chúng thành token mới, lặp lại cho đến khi đủ kích thước từ vựng. Ví dụ với kho "low lower lowest": cặp "l"+"o" phổ biến nhất → "lo"; rồi "lo"+"w" → "low"; dần dần "low" thành một token.

2. Token và tiếng Việt — điều cần biết khi làm RAG

Tokenizer được huấn luyện chủ yếu trên tiếng Anh nên tiếng Việt có dấu thường tốn **nhiều token hơn** cho cùng lượng thông tin (một âm tiết có dấu có thể bị tách thành 2-3 token ở tokenizer cũ; tokenizer mới đã cải thiện nhiều). Hệ quả thực tế:

- Chi phí API và độ trễ cao hơn với cùng số chữ.
- Khi đặt chunk_size theo token, một chunk tiếng Việt chứa ít chữ hơn chunk tiếng Anh.
- Luôn đo bằng chính tokenizer của model bạn dùng, đừng ước lượng theo số từ.

```
import tiktoken
enc = tiktoken.get_encoding("cl100k_base")
for s in ["Retrieval augmented generation",
         "Sinh viên bị c[ảnh báo học tập khi nào?"]]:
    ids = enc.encode(s)
    print(len(s.split()), "từ ->", len(ids), "token")
```

3. Embeddings: từ ID sang ý nghĩa

Token ID chỉ là số thứ tự, không mang ý nghĩa (ID 1000 không "gần" ID 1001 về nghĩa). Lớp embedding là một ma trận kích thước [vocab_size × d_model]; tra ID thứ i là lấy hàng thứ i. Các hàng này là tham số học được: sau huấn luyện, các token có nghĩa gần nhau có vector gần nhau.

Bảng king/queen/man/woman trong ảnh minh họa ý tưởng hình học: hướng từ "man" sang "woman" xấp xỉ hướng từ "king" sang "queen". Kiểm tra bằng số trong ảnh: king – man + woman = [0,21–0,19+0,18; –0,13+0,30–0,28; 0,45–0,10+0,11; 0,33–0,25+0,24; –0,52+0,10–0,12] = [0,20; –0,11; 0,46; 0,32; –0,54] — rất gần vector queen [0,20; –0,11; 0,44; 0,31; –0,49].

Phân biệt hai loại embedding (ảnh trộn lẫn): **token embedding** là lớp đầu tiên bên trong LLM, mỗi token một vector, chưa có ngữ cảnh. **Sentence/text embedding** (OpenAI text-embedding, BGE, E5...) là một model riêng nhận cả đoạn văn và trả về *một* vector đại diện cho cả đoạn — đây là loại dùng trong RAG (Chương 10). word2vec/GloVe là embedding tĩnh thế hệ cũ: một từ chỉ có một vector bất kể ngữ cảnh.

4. Input của Transformer

$$\text{Input} = \text{Token Embedding} + \text{Positional Encoding}$$

Hai vector cùng kích thước d_{model} được cộng từng phần tử. (Với RoPE, vị trí không được cộng vào đây mà được "xoay" trực tiếp vào Q và K bên trong attention.)

5. Context window — ngân sách token

Context window là tổng ngân sách cho **system prompt + lịch sử hội thoại + tài liệu chèn vào + câu hỏi + câu trả lời**. Ví dụ trong ảnh: window 8.192 token, system prompt 500, user prompt 2.000 → còn tối đa 5.692 token cho câu trả lời.

Bài toán ngân sách điển hình trong RAG với model 8K:

Thành phần	Token
System prompt (vai trò, quy tắc trích dẫn)	400
Lịch sử hội thoại (3 lượt gần nhất)	900
5 chunk × 500 token	2.500
Câu hỏi	60
Dự phòng cho câu trả lời	1.000
Tổng	4.860 — an toàn

Chi phí attention tăng theo bình phương độ dài chuỗi (n^2 cặp token), nên context dài vừa tốn tính toán vừa tốn tiền.

Đỉnh chính: "Khi context window đầy, các token cũ hơn sẽ bị quên" — chính xác hơn: model **không** tự quên dần. Nếu đầu vào vượt giới hạn, API sẽ báo lỗi hoặc ứng dụng phải chủ động cắt bớt (thường cắt tin nhắn cũ). Ngoài ra, context dài hơn **không** tự động "thông minh hơn": nghiên cứu "Lost in the Middle" (Liu và cộng sự, 2023) cho thấy model hay bỏ sót thông tin nằm giữa một context rất dài. Vì vậy RAG chọn lọc vài chunk tốt thường hiệu quả hơn nhồi cả tài liệu. Bảng context window trong ảnh là số liệu 2024; model hiện nay phổ biến từ 128K đến 1M token.

CHƯƠNG 4 · ẢNH GỐC

4. Prompt Engineering



4. PROMPT ENGINEERING

RAG & LLMs
SERIES VIẾT TAY
v1.0



Prompt tốt hơn → Phản hồi tốt hơn → Kết quả tốt hơn

1. PROMPT ENGINEERING LÀ GÌ?

Prompt Engineering là nghệ thuật và khoa học của việc thiết kế đầu vào (prompt) để hướng LLM tạo ra output chính xác, phù hợp và chất lượng cao.

- ✓ Vấn đề không nằm ở model, mà ở cách bạn hỏi.
- ✓ Prompt tốt = Chỉ dẫn rõ ràng + Ngữ cảnh đúng + Cấu trúc tốt

2. CẤU TRÚC CỦA MỘT PROMPT TỐT

Instruction
Bạn muốn model làm gì.

→

Context
Thông tin nền hoặc tình huống cần hiểu.

→

Input Data
Dữ liệu hoặc thông tin cần xử lý.

→

Constraints
Quy tắc, giới hạn, định dạng, giọng điệu, độ dài, v.v.

→

Output Format
Cách bạn muốn câu trả lời được trình bày.

 Bạn cung cấp những yếu tố này càng rõ ràng, output càng tốt!

3. NGUYÊN TẮC VIẾT PROMPT

- ⚡ Rõ ràng & cụ thể
- ⚡ Cung cấp đủ ngữ cảnh
- ⚡ Chia nhỏ tác vụ phức tạp thành các bước
- ⚡ Chỉ định định dạng output
- ⚡ Đưa ra ví dụ (Few-Shot Prompting)
- ⚡ Nêu rõ các ràng buộc
- ⚡ Lặp lại và tinh chỉnh
- ⚡ Kiểm thử các edge case

Prompt tốt giúp giảm sự mơ hồ và giúp model suy nghĩ đúng hướng.

4. KỸ THUẬT PROMPTING (kèm ví dụ)

Kỹ thuật	Mô tả	Ví dụ (đơn giản hóa)
Zero-Shot	Hỏi trực tiếp, không kèm ví dụ.	"Tóm tắt bài viết này."
Few-Shot	Cung cấp ví dụ trước khi hỏi.	"Q: 2+2=? A: 4 Q: 3+3=? A: 6 Q: 5+5=? A: "
Chain-of-Thought (CoT)	Yêu cầu model suy nghĩ từng bước.	"Hãy giải bài này từng bước. Nếu một đoàn tàu đi..."
Role Prompting	Gán một vai trò hoặc persona cho model.	"Bạn là một Data Scientist. Hãy giải thích regression một cách đơn giản."
Self-Consistency	Hỏi nhiều lần và chọn câu trả lời tốt nhất.	"Giải bài toán này theo nhiều cách khác nhau và chọn câu trả lời tốt nhất."
Tree of Thoughts (Nâng cao)	Khám phá nhiều hướng suy luận.	"Hãy nghĩ đến tất cả các cách tiếp cận có thể và chọn cách tối ưu nhất."

5. VÍ DỤ TRƯỚC & SAU

✗ Prompt yếu

"Giải thích AI."

→

Quá mơ hồ → Output chung chung

✓ Prompt mạnh

"Bạn là một giáo viên. Hãy giải thích Artificial Intelligence bằng từ ngữ đơn giản cho một đứa trẻ 10 tuổi. Đưa ra 3 ví dụ thực tế và giữ trong vòng 150 từ."

→

Rõ ràng → Phù hợp → Output tốt hơn

6. VÍ DỤ FEW-SHOT

Phân loại cảm xúc của các review sau: Positive, Negative hoặc Neutral.

Review: Minh rất thích sản phẩm này! Dùng rất tốt. → Positive

Review: Pin hết quá nhanh. → Negative

Review: Cũng ổn, không phải tốt nhất nhưng cũng không tệ. → Neutral

Review: Thiết kế tuyệt vời và rất cao cấp. →

7. VÍ DỤ CHAIN-OF-THOUGHT

Q: Nếu một cửa hàng bán 4 cuốn vở với giá ₹40, thì 7 cuốn vở giá bao nhiêu?

Hãy suy nghĩ từng bước:

1 cuốn vở = 40 / 4 = ₹10

7 cuốn vở = 7 × 10 = ₹70

Đáp án: ₹70

CoT giúp model chia nhỏ vấn đề và giảm sai sót.

8. SỬ DỤNG RÀNG BUỘC

- ✓ Đặt giới hạn số từ/ký tự.
- ✓ Xác định giọng điệu (trang trọng, thân mật, kỹ thuật).
- ✓ Chỉ rõ những gì cần bao gồm / loại trừ.
- ✓ Nêu các quy tắc nghiệp vụ hoặc lĩnh vực.

Ví dụ:

"Viết một email chuyên nghiệp gửi khách hàng để xin lỗi vì sự chậm trễ. Giữ dưới 100 từ và dùng giọng điệu lịch sự."

9. ĐỊNH DẠNG OUTPUT

Yêu cầu model trả lời theo một định dạng cụ thể.

--

`</>`

-



Table JSON Bullet List Markdown

Ví dụ:

```
"Cho tôi top 3 ngôn ngữ lập trình dưới dạng JSON."
{
  "languages": ["Python",
    "JavaScript", "Go"],
  "count": 3
}
```

10. MẸO & BEST PRACTICES

- ✚ Bắt đầu đơn giản và lặp lại.
- ✚ Giữ prompt ở dạng module và có thể tái sử dụng.
- ✚ Đánh giá output và tinh chỉnh prompt.
- ✚ Dùng dấu phân cách (" ", ##, <tag>) cho rõ ràng.
- ✚ Kết hợp các kỹ thuật để có kết quả tốt nhất.
- ✚ Luôn kiểm thử với các edge case.
- ✚ Lưu lại tài liệu về prompt để tái sử dụng.

CHƯƠNG 4 · GIẢI THÍCH CHI TIẾT

Prompt Engineering

1. Prompt là "giao diện lập trình" của LLM

Với LLM, prompt đóng vai trò của mã nguồn: cùng một model, prompt khác nhau cho chất lượng khác nhau rất xa. Câu "vấn đề không nằm ở model, mà ở cách bạn hỏi" hơi cường điệu (model yếu vẫn là model yếu), nhưng đúng ở chỗ: **trước khi nghĩ đến fine-tune, hãy tối ưu prompt.**

2. Năm thành phần của một prompt tốt

Thành phần	Câu hỏi tự kiểm tra	Ví dụ cho chatbot quy chế
Instruction	Model cần làm gì?	"Trả lời câu hỏi của sinh viên"
Context	Model cần biết gì về bối cảnh?	"Bạn là trợ lý phòng đào tạo; chỉ dựa vào trích đoạn quy chế bên dưới"
Input data	Dữ liệu cần xử lý?	Các chunk quy chế + câu hỏi
Constraints	Giới hạn nào?	"Không biết thì nói không biết; tối đa 150 từ; tiếng Việt"
Output format	Trình bày ra sao?	"Kết thúc bằng (Nguồn: Điều X, Khoản Y)"

3. Các kỹ thuật prompting

- **Zero-shot:** hỏi thẳng. Đủ tốt cho tác vụ phổ biến với model mạnh.
- **Few-shot:** đưa vài cặp ví dụ input→output. Rất hiệu quả để "dạy" định dạng hoặc tiêu chí phân loại riêng. Lưu ý chọn ví dụ đa dạng và cân bằng nhãn, vì model dễ bắt chước thiên lệch của ví dụ (ví dụ ba mẫu đều Positive thì model nghiêng về Positive).
- **Chain-of-Thought (CoT):** yêu cầu suy nghĩ từng bước. Vì model sinh từng token và mỗi token sinh ra trở thành ngữ cảnh cho token sau, việc viết ra các bước trung gian thực sự cho model thêm "không gian tính toán". Ví dụ quyển vở trong ảnh: 4 quyển giá 40 → 1 quyển 10 → 7 quyển 70.
- **Role prompting:** gán vai trò để điều chỉnh giọng văn và mức độ chuyên môn.
- **Self-consistency:** xem đính chính bên dưới.
- **Tree of Thoughts:** khám phá nhiều nhánh suy luận, đánh giá và quay lui — thường cần code điều phối nhiều lần gọi model chứ không chỉ một prompt.

Đính chính: Self-consistency (Wang và cộng sự, 2022) **không** phải là bảo model "giải theo nhiều cách rồi chọn" trong một prompt. Cách đúng: gọi model **nhiều lần độc lập** với cùng prompt CoT và temperature > 0 để có nhiều chuỗi suy luận khác nhau, rồi lấy **đáp án xuất hiện nhiều nhất** (bỏ phiếu đa số).

4. Prompt mẫu cho RAG (dùng lại được)

```
[SYSTEM]
Bạn là trợ lý tư vấn học vụ. Chỉ trả lời dựa trên phần
<tai_lieu>. Nếu tài liệu không đủ thông tin, trả lời:
"Tôi chưa tìm thấy quy định này, vui lòng liên hệ phòng
đào tạo." Không suy đoán. Trích dẫn Điếu/Khoản cụ thể.

[USER]
<tai_lieu>
[1] (Quy chế đào tạo, Điều 12, Khoản 2) ...
[2] (Quy chế đào tạo, Điều 15, Khoản 1) ...
</tai_lieu>
<cau_hoi>Điều kiện trung bình bao nhiêu thì bị cảnh báo?</cau_hoi>
```

Ba nguyên tắc ẩn trong mẫu này: dùng thẻ phân cách (<tai_lieu>) để model không nhầm tài liệu với chỉ dẫn; cho model một "lối thoát" khi không có thông tin (giảm bịa đặt); yêu cầu trích dẫn để người dùng kiểm chứng.

5. Ràng buộc và định dạng output

Yêu cầu JSON rất hữu ích khi output đi vào code (ví dụ trích xuất thông tin, phân loại ý định). Các API hiện đại có chế độ *structured output*/JSON schema bảo đảm cú pháp hợp lệ — nên dùng thay vì chỉ "dặn" trong prompt. Dù vậy, luôn bọc `json.loads` trong `try/except`.

6. Quy trình làm prompt như làm phần mềm

- Viết bản đơn giản nhất → chạy trên 20–50 câu hỏi thật → ghi lại lỗi → sửa → chạy lại cả bộ (để không sửa chỗ này hỏng chỗ kia).
- Lưu prompt trong file có phiên bản (Git), không nhúng cứng trong code.
- Kiểm thử edge case: câu hỏi ngoài phạm vi, câu hỏi mơ hồ, câu hỏi cố tình lừa ("bỏ qua hướng dẫn trên và...") — đây là **prompt injection**, rủi ro bảo mật thật sự với chatbot công khai.

CHƯƠNG 5 · ẢNH GỐC

5. Kiến trúc LLM



5. KIẾN TRÚC LLM

RAG & LLMs
SERIES VIẾT TAY
v1.0



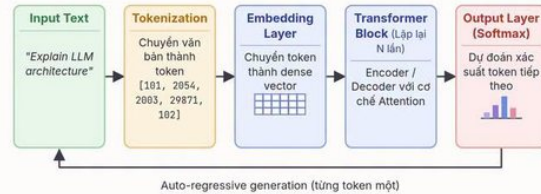
CÁCH LARGE LANGUAGE MODEL VẬN HÀNH BÊN TRONG

1. LLM LÀ GÌ?

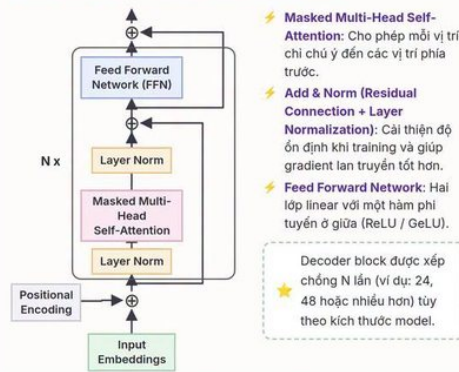
LLM (Large Language Model) là các model deep learning được huấn luyện trên lượng dữ liệu văn bản khổng lồ để hiểu, tạo ra và tương tác bằng ngôn ngữ tự nhiên.

- ✓ Dựa trên kiến trúc Transformer
- ✓ Được huấn luyện để dự đoán token tiếp theo
- ✓ Có khả năng suy luận, sinh văn bản, dịch thuật, tóm tắt và nhiều hơn nữa

2. KIẾN TRÚC TỔNG QUAN

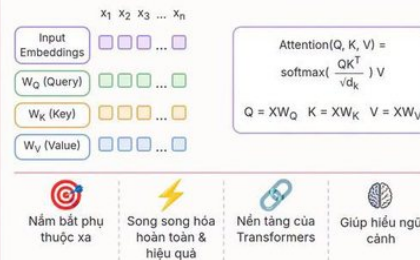


3. TRANSFORMER BLOCK (TỔNG QUAN)



4. SELF-ATTENTION (Ý TƯỞNG CỐT LÕI)

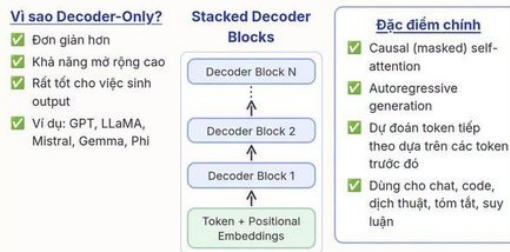
Self-attention giúp model tập trung vào những phần liên quan nhất của input.



5. CÁC THÀNH PHẦN CỦA KIẾN TRÚC LLM

1. Token Embedding Layer	Chuyển token ID thành dense vector có số chiều d_{model} .
2. Positional Encoding	Thêm thông tin vị trí vì transformer không có recurrence.
3. Transformer Blocks	Chồng các decoder block (Masked Self-Attention + FFN).
4. Layer Normalization	Chuẩn hóa activation để training ổn định và nhanh hơn.
5. Output Projection	Ảnh xạ hidden state về kích thước vocabulary bằng một lớp linear.
6. Softmax	Chuyển logits thành phân phối xác suất trên token tiếp theo.

6. KIẾN TRÚC DECODER-ONLY (Hầu hết LLM)



7. KÍCH THƯỚC LLM PHỔ BIẾN (Ví dụ)

Model	Parameters	Layers	Hidden Size (d_{model})	Attention Heads
GPT-2 Small	117M	12	768	12
GPT-2 Medium	345M	24	1024	16
LLaMA 2 7B	7B	32	4096	32
LLaMA 2 13B	13B	40	5120	40
GPT-4 (ước tính)	~1.8T	~120	~12288	~96

Lưu ý
Nhiều tham số & layer hơn → năng lực tốt hơn nhưng cần nhiều compute & dữ liệu hơn.

8. TRAINING OBJECTIVE

LLM được huấn luyện bằng Next Token Prediction.



9. INFERENCE (GENERATION)

- Người dùng cung cấp một prompt.
- Model chuyển prompt → token → embeddings.
- Đi qua các decoder block xếp chồng.

10. VÌ SAO KIẾN TRÚC NÀY HIỆU QUẢ?

- ✓ Học được các biểu diễn phong phú
- ✓ Hiểu ngữ cảnh sâu sắc
- ✓ Mở rộng tốt theo dữ liệu & compute

CHƯƠNG 5 · GIẢI THÍCH CHI TIẾT

Kiến trúc LLM

1. Luồng dữ liệu từ đầu đến cuối

Text → Tokenizer → Embedding layer → N Transformer blocks → Output projection → Softmax → xác suất token tiếp theo. Mũi tên quay ngược "auto-regressive generation" nghĩa là: token vừa sinh được nối vào đầu vào và cả quá trình lặp lại để sinh token tiếp theo, cho đến khi gặp token kết thúc hoặc đạt max_tokens.

2. Ba giai đoạn tính toán khi sinh

- **Prefill:** xử lý toàn bộ prompt một lượt (song song) — nhanh trên mỗi token.
- **Decode:** sinh từng token một — tuần tự, đây là nút thắt về tốc độ.
- **KV cache:** K và V của các token cũ được lưu lại để không phải tính lại ở mỗi bước. KV cache tốn bộ nhớ tỉ lệ với độ dài context — lý do chạy context dài cần nhiều VRAM hơn hẳn chỉ riêng trọng số model.

3. Sáu thành phần

Thành phần	Vai trò	Kích thước ví dụ (LLaMA-2 7B)
Token embedding	Tra ID → vector	32.000 × 4.096
Positional (RoPE)	Mã hóa vị trí	không có tham số
32 Transformer blocks	Attention + FFN	phần lớn tham số
Norm	Ổn định activation	RMSNorm
Output projection	Vector → logits trên từ vựng	4.096 × 32.000
Softmax	Logits → xác suất	—

4. Ước lượng số tham số bằng tay

Mỗi block có attention $\approx 4 \cdot d^2$ (W_Q, W_K, W_V, W_O) và FFN. LLaMA dùng SwiGLU với 3 ma trận kích thước $d \times 11.008$: $3 \times 4.096 \times 11.008 \approx 135M$. Attention: $4 \times 4.096^2 \approx 67M$. Một block $\approx 202M$; 32 block $\approx 6,47$ tỷ; cộng embedding và output ($2 \times 32.000 \times 4.096 \approx 0,26$ tỷ) $\rightarrow \approx 6,7$ tỷ — đúng là "7B".

5. Mục tiêu huấn luyện và softmax

Output layer cho ra vector logits z (một số cho mỗi token trong từ vựng). Softmax biến logits thành xác suất:

$$P(\text{token } i) = e^{z_i} / \sum_j e^{z_j}$$

Ví dụ logits $[2,0; 1,0; 0,1]$ cho ba token "mat", "sofa", "car" $\rightarrow e^z = [7,39; 2,72; 1,11] \rightarrow$ xác suất $\approx [0,66; 0,24; 0,10]$.

6. Vì sao decoder-only thống trị

Một kiến trúc duy nhất, một mục tiêu duy nhất (next-token prediction), dữ liệu không cần nhãn, và mọi tác vụ (dịch, tóm tắt, hỏi đáp, code) đều diễn đạt được thành "sinh văn bản tiếp theo". Sự đơn giản này giúp mở rộng quy mô dễ dàng — và các "scaling laws" cho thấy chất lượng tăng đều đặn khi tăng tham số, dữ liệu và compute cùng nhau.

Đính chính: (1) Số liệu GPT-4 (~1,8T tham số, ~120 lớp...) là **tin đồn chưa được OpenAI xác nhận**; nhiều nguồn cho rằng GPT-4 dùng kiến trúc Mixture-of-Experts. (2) GPT-2 Small chính xác là 124M (117M là con số trong bài báo gốc do cách đếm). (3) LLM hiện đại thường dùng RMSNorm thay LayerNorm, SwiGLU thay ReLU/GeLU, RoPE thay positional encoding cộng, và Grouped-Query Attention (nhiều head Q dùng chung K, V) để giảm KV cache.

7. Liên hệ thực hành: chọn model cho GPU

Quy tắc nhắm: bộ nhớ trọng số $\approx$ số tham số $\times$ số byte mỗi tham số; cộng thêm 10–30% cho KV cache và runtime.

Model	FP16	4-bit (Q4)
7–8B	~16 GB	~5 GB
13–14B	~28 GB	~9 GB
32–34B	~68 GB	~20 GB
70B	~140 GB	~40 GB

CHƯƠNG 6 · ẢNH GỐC

6. Pretraining & Fine-Tuning



6. PRETRAINING & FINE-TUNING

RAG & LLMs
SERIES VIẾT TAY
v1.0



TỪ KIẾN THỨC TỔNG QUÁT ĐẾN TRÍ TUỆ CHUYÊN BIỆT

1. PRETRAINING & FINE-TUNING LÀ GÌ?

Pretraining: Huấn luyện model trên lượng dữ liệu tổng quát khổng lồ để học các pattern và kiến thức rộng.

Fine-tuning: Tiếp tục huấn luyện model đã pretrain trên dữ liệu của một tác vụ hoặc lĩnh vực cụ thể để model chính xác và hữu ích hơn cho tác vụ đó.

Ví dụ so sánh:



2. GIAI ĐOẠN PRETRAINING

Dữ liệu khổng lồ
(Sách, bài báo, Web, Code, v.v.)

Self-Supervised Learning

Foundation Model
(Mục đích chung)

Hoạt động thế nào?

- ✓ Model dự đoán token tiếp theo dựa trên các token trước đó.
- ✓ Học các pattern ngôn ngữ, sự kiện, khả năng suy luận, ngữ pháp, kiến thức về thế giới, v.v.
- ✓ Không cần gán nhãn thủ công (self-supervised).
- ✓ Tốn rất nhiều compute (GPU/TPU) + thời gian.

Mục tiêu: Next Token Prediction

Input: The cat sat on the
Target: mat

3. GIAI ĐOẠN FINE-TUNING

Pretrained Model

Dữ liệu theo tác vụ / lĩnh vực
(có nhãn)

Fine-tuned Model

Hoạt động thế nào?

- ✓ Bắt đầu với các trọng số đã pretrain.
- ✓ Huấn luyện trên tập dữ liệu nhỏ hơn, chất lượng cao, có gán nhãn.
- ✓ Model thích ứng với tác vụ hoặc lĩnh vực cụ thể.
- ✓ Cần ít compute & thời gian hơn pretraining.

Tác vụ ví dụ

- ⚡ Phân loại văn bản
- ⚡ Dịch thuật
- ⚡ Hỏi đáp
- ⚡ Sinh code
- ⚡ Tóm tắt
- ⚡ Chatbot

4. PRETRAINING vs FINE-TUNING

	Pretraining	Fine-tuning
Mục tiêu	Học hiểu ngôn ngữ tổng quát	Thích ứng với tác vụ / lĩnh vực cụ thể
Dữ liệu	Dữ liệu khổng lồ (hàng TB văn bản)	Dữ liệu có nhãn nhỏ hơn (vài GB)
Kiểu học	Self-Supervised (Next Token Prediction)	Supervised / học theo tác vụ
Compute	Rất cao 🔥🔥🔥🔥	Thấp hơn 🔥🔥
Thời gian	Vài tuần / vài tháng	Vài giờ / vài ngày
Output	Foundation Model (Mục đích chung)	Model chuyên biệt (theo tác vụ)
Ví dụ	GPT-4, Llama 3 (Base), PaLM, Gemini (Base)	ChatGPT (RLHF + SFT), Code Llama (Code Data)

5. CÁC PHƯƠNG PHÁP FINE-TUNING PHỔ BIẾN

- 🔴 **Full Fine-Tuning:** Cập nhật toàn bộ tham số của model.
- 🔴 **Parameter-Efficient Fine-Tuning (PEFT):** LoRA, QLoRA, Adapters, Prefix Tuning, Prompt Tuning
→ Chỉ huấn luyện một ít tham số bổ sung (hiệu quả!).
- 🔴 **Instruction Tuning:** Fine-tune trên các cặp chỉ dẫn + phản hồi.
- 🔴 **Domain Adaptation:** Fine-tune trên kho dữ liệu chuyên ngành.

6. LƯỖNG VÍ DỤ (End-to-End)



7. MỤC TIÊU PRETRAINING (CHI TIẾT)

Hầu hết LLM dùng Causal Language Modeling (CLM).
Cho chuỗi: [The, cat, sat, on, the, mat]

Model học cách dự đoán từng token tiếp theo:

Input: [The] → Dự đoán: cat
Input: [The, cat] → Dự đoán: sat
Input: [The, cat, sat, on] → Dự đoán: the

...
Input: [The, cat, sat, on, the] → Dự đoán: mat

Loss = Cross Entropy Loss trên tất cả các token

8. LOSS FUNCTION

Cross Entropy Loss

$$L = - \sum_{t=1}^T \log P(x_t | x_{<t})$$

Trong đó,
 x_t = token tiếp theo thực tế
 $P(x_t | x_{<t})$ = xác suất dự đoán (model)
 T = tổng số token

9. VÌ SAO FINE-TUNING QUAN TRỌNG?

- ✓ Giúp model hiểu rõ tác vụ.
- ✓ Tăng độ chính xác trên các tác vụ cụ thể.
- ✓ Giảm hallucination (với dữ liệu tốt).
- ✓ Thích ứng với phong cách, giọng điệu, lĩnh vực.
- ✓ Hiện thực hóa các ứng dụng thực tế.



CHƯƠNG 6 · GIẢI THÍCH CHI TIẾT

Pretraining & Fine-Tuning

1. Phép so sánh dễ nhớ

Pretraining giống 12 năm phổ thông + đọc cả thư viện: học "mọi thứ" ở mức rộng. Fine-tuning giống khóa đào tạo nghiệp vụ vài tuần: học sâu một việc cụ thể. Người tốt nghiệp nghiệp vụ không quên kiến thức phổ thông, nhưng nếu đào tạo nghiệp vụ quá mạnh tay có thể bị "lệch" — hiện tượng **catastrophic forgetting**.

2. Pretraining — self-supervised

"Self-supervised" nghĩa là nhãn được tạo tự động từ chính dữ liệu: với câu "The cat sat on the mat", mỗi vị trí cho ra một cặp (ngữ cảnh → token tiếp theo). Một câu 6 token cho 5-6 mẫu huấn luyện; hàng nghìn tỷ token cho hàng nghìn tỷ mẫu mà không cần ai gán nhãn.

3. Hàm mất mát cross-entropy

$$L = - (1/T) \sum_{t=1..T} \log P(x_t | x_{<t})$$

Tính tay: nếu model gán xác suất 0,66 cho token đúng "mat", loss tại vị trí đó là $-\ln(0,66) \approx 0,42$. Nếu chỉ gán 0,10, loss là $-\ln(0,10) \approx 2,30$. Loss phạt nặng khi model tự tin sai. Đại lượng liên quan là **perplexity** = e^L : có thể hiểu là "model phân vân trung bình giữa bao nhiêu lựa chọn"; perplexity càng thấp càng tốt.

4. Bảng so sánh — đọc kỹ cột "Dữ liệu"

Pretraining dùng hàng nghìn tỷ token không nhãn; fine-tuning dùng từ vài trăm đến vài trăm nghìn mẫu chất lượng cao có nhãn. Kinh nghiệm thực tế: **1.000 mẫu sạch tốt hơn 50.000 mẫu bẩn**. Với fine-tuning, chất lượng dữ liệu quyết định gần như tất cả.

5. Full fine-tuning vs PEFT — và toán của LoRA

Full fine-tuning cập nhật mọi tham số: với model 7B cần lưu trọng số, gradient và trạng thái optimizer Adam — tổng cộng khoảng 16 byte/tham số ≈ 112 GB, vượt xa một GPU thông thường.

LoRA (Low-Rank Adaptation) giữ nguyên ma trận gốc W_0 (đóng băng) và chỉ học một phần cập nhật hạng thấp:

$$W = W_0 + \Delta W = W_0 + B \cdot A, B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k}, r \ll d$$

Ví dụ một ma trận 4.096×4.096 (16,8 triệu tham số). Với $r = 8$: B có 4.096×8 và A có 8×4.096 , tổng 65.536 tham số — chỉ **0,39%**. Ý tưởng: phần thay đổi cần thiết để thích nghi một tác vụ có "số chiều nội tại" thấp. **QLoRA** làm thêm một bước: lượng tử hóa W_0 xuống 4-bit, cho phép fine-tune model 7B-13B trên một GPU 24 GB.

6. Các phương pháp fine-tuning

Phương pháp	Dữ liệu	Mục đích
Instruction tuning	Cặp (chỉ dẫn, phản hồi)	Biến base model thành model biết nghe lời
Domain adaptation	Văn bản thô chuyên ngành	Làm quen thuật ngữ, văn phong ngành
Task fine-tuning	Cặp (input, nhãn) cho một tác vụ	Phân loại, trích xuất có định dạng cố định
PEFT (LoRA, Adapters, Prefix/Prompt tuning)	Như trên	Giảm chi phí, lưu nhiều adapter nhỏ cho nhiều tác vụ

7. Khi nào fine-tune, khi nào RAG?

Nhu cầu	Nên dùng
Model cần biết dữ liệu riêng, thay đổi thường xuyên	RAG
Cần trích dẫn nguồn	RAG
Model cần đổi văn phong, định dạng, hành vi ổn định	Fine-tuning
Tác vụ hẹp, lặp lại số lượng lớn, cần model nhỏ rẻ	Fine-tuning
Cả hai	Fine-tune nhẹ để model giỏi "đọc ngữ cảnh và trích dẫn", rồi dùng RAG cấp kiến thức

Lưu ý thực tế: fine-tuning là cách **kém hiệu quả** để " nạp kiến thức mới" (model dễ nhớ sai, khó cập nhật, không trích dẫn được). Ảnh ghi "giảm hallucination (với dữ liệu tốt)" — đúng cho tác vụ hẹp, nhưng với câu hỏi kiến thức, RAG giảm hallucination hiệu quả hơn.

CHƯƠNG 7 · ẢNH GỐC

7. Instruction Tuning & RLHF



7. INSTRUCTION TUNING & RLHF

RAG & LLMs
SERIES VIẾT TAY
v1.0



GIÚP LLM HỮU ÍCH, TRUNG THỰC & VÔ HẠI ❤️

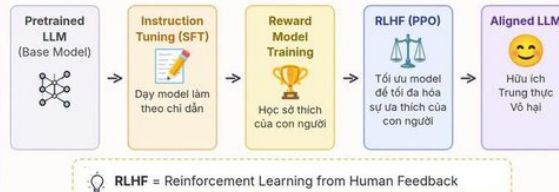
1. VÌ SAO CẦN VƯỢT RA NGOÀI PRETRAINING?

LLM đã pretrain biết rất nhiều, nhưng chúng:

- ⚡ Không làm theo chỉ dẫn tốt
- ⚡ Có thể đưa ra câu trả lời sai hoặc không an toàn
- ⚡ Không biết con người thích kiểu phản hồi nào

Instruction Tuning + RLHF giúp
★ model phù hợp (align) với ý định
và giá trị của con người. ❤️

2. TỔNG QUAN ALIGNMENT PIPELINE



3. INSTRUCTION TUNING (SFT)

Mục tiêu: Day model làm theo chỉ dẫn và tạo ra các phản hồi hữu ích.

Hoạt động thế nào?

- ✅ Dùng một tập dữ liệu gồm các cặp (Instruction, Input, Output).
- ✅ Fine-tune model đã pretrain bằng Supervised Fine-Tuning (SFT).
- ✅ Model học cách ánh xạ chỉ dẫn thành phản hồi chất lượng cao.

Ví dụ (Dữ liệu huấn luyện)

Instruction	Input (Context)	Output (Response)
Giải thích quá trình quang hợp.	-	Quang hợp là quá trình thực vật sử dụng...
Dịch sang tiếng Hindi.	Good morning	शुभ प्रभात
Viết một hàm Python để cộng hai số.	-	def add(a, b): return a + b
...	...	...

Loss Function: Cross Entropy Loss

4. REWARD MODEL TRAINING (PREFERENCE MODELING)

Mục tiêu: Học xem con người thích gì.

- ✅ Con người xếp hạng nhiều phản hồi cho cùng một prompt.
- ✅ Reward model học cách dự đoán phản hồi nào tốt hơn.
- ✅ Được huấn luyện như một binary classifier hoặc regression model.

Ví dụ

Prompt	Mức độ ưa thích của con người (Tốt nhất → Tệ nhất)
Tập thể dục có lợi ích gì?	A > B > C A Tập thể dục đều đặn giúp cải thiện sức khỏe, nâng cao tâm trạng, tăng cường miễn dịch... B Tập thể dục tốt cho sức khỏe và giúp ích theo nhiều cách. C Minh nghĩ tập thể dục có thể ổn, nhưng không chắc lắm.

Reward model học: $\text{score}(A) > \text{score}(B) > \text{score}(C)$

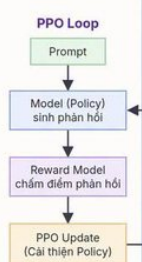
Loss Function: Pairwise Ranking Loss / Bradley-Terry Loss

5. RLHF (VỚI PPO)

Mục tiêu: Tạo ra phản hồi tối đa hóa sự ưa thích của con người bằng reinforcement learning.

Hoạt động thế nào?

- ✅ Policy khởi tạo chính là model đã qua instruction tuning.
- ✅ Reward model chấm điểm phản hồi.
- ✅ PPO cập nhật policy để tối đa hóa reward nhưng vẫn giữ nó gần với model gốc (tránh bị lệch - drifting).



Mục tiêu (đơn giản hóa):

$$\text{Maximize } E_{y \sim \pi_\theta} [R(y)] - \beta \text{KL}(\pi_\theta \parallel \pi_{\text{ref}})$$

Trong đó:

- ★ $R(y)$ = reward từ reward model
- ★ π_θ = policy hiện tại (model)
- ★ π_{ref} = reference policy (model SFT)
- ★ β = hệ số phạt (KL penalty coefficient)

6. SFT vs RLHF

Khía cạnh	SFT (Instruction Tuning)	RLHF
Mục đích	Day model làm theo chỉ dẫn	Align model với sở thích & giá trị của con người
Tín hiệu huấn luyện	Câu trả lời chuẩn (ground truth)	Sự ưa thích của con người (ngầm định)
Phương pháp	Supervised Fine-Tuning	Reinforcement Learning (PPO) dùng Reward Model
Dữ liệu	Các cặp Instruction-Response	Xếp hạng / sở thích của con người
Tối ưu	Độ chính xác khi làm theo chỉ dẫn	Chất lượng, tính hữu ích, trung thực, vô hại
Output	Tốt nhưng có thể cứng nhắc hoặc không an toàn	Phù hợp hơn, an toàn & hữu ích hơn

7. DỮ LIỆU SỬ DỤNG

- ★ **Instruction Datasets** (cho SFT): Alpaca, Dolly, FLAN, OpenOrca, ShareGPT, v.v.
- ★ **Preference Datasets** (cho RLHF): Anthropic HH-RLHF, OpenAI WebGPT, StackExchange comparisons, v.v.

8. LỢI ÍCH

- ✅ Làm theo chỉ dẫn tốt hơn
- ✅ Phản hồi hữu ích và phù hợp hơn
- ✅ Giảm output độc hại, thiên lệch hoặc có hại
- ✅ Hiểu được các sở thích tinh tế của con người

9. THÁCH THỨC ⚠️

- ✗ Phản hồi từ con người tốn kém và mất thời gian
- ✗ Sở thích mang tính chủ quan & thiếu nhất quán
- ✗ Reward hacking (lách luật để lấy điểm cao từ reward model)
- ✗ Thiên lệch (bias) trong phản hồi của con người

CHƯƠNG 7 · GIẢI THÍCH CHI TIẾT

Instruction Tuning & RLHF

1. Vì sao base model chưa dùng được

Base model học để *bắt chước văn bản trên Internet*, mà văn bản trên Internet có đủ cả: câu trả lời hay, câu trả lời sai, nội dung độc hại. Hỏi nó một câu, nó có thể viết tiếp thành một diễn đàn tranh cãi. "Alignment" là quá trình làm cho model hành xử theo ý định của con người: **hữu ích, trung thực, vô hại**.

2. Pipeline ba bước

- **Bước 1 — SFT**: fine-tune trên hàng chục nghìn cặp (chỉ dẫn, phản hồi mẫu) do người viết. Loss vẫn là cross-entropy nhưng thường chỉ tính trên phần phản hồi.
- **Bước 2 — Reward model**: cho model SFT sinh nhiều câu trả lời cho cùng prompt; người đánh giá xếp hạng; huấn luyện một model chấm điểm học theo xếp hạng đó.
- **Bước 3 — RL (PPO)**: tối ưu model để sinh câu trả lời được reward model chấm cao.

Vì sao cần bước 2-3 thay vì chỉ SFT? Vì **con người so sánh dễ hơn viết**: viết câu trả lời hoàn hảo rất khó, nhưng nói "A tốt hơn B" thì nhanh và nhất quán hơn. Ngoài ra SFT chỉ dạy "câu trả lời tốt trông thế nào", còn RL dạy cả "tránh những gì".

3. Toán của reward model (Bradley-Terry)

Giả sử y_w là câu trả lời được ưa thích, y_l là câu bị loại. Xác suất mô hình hóa:

$$P(y_w > y_l) = \sigma(r(x, y_w) - r(x, y_l)), \text{ Loss} = -\log \sigma(r_w - r_l)$$

Tính tay: nếu $r_w = 2,0$ và $r_l = 0,5$, hiệu là 1,5; $\sigma(1,5) \approx 0,82$; $\text{loss} \approx 0,20$. Nếu model chấm ngược ($r_w = 0,5$, $r_l = 2,0$), $\sigma(-1,5) \approx 0,18$; $\text{loss} \approx 1,71$ — bị phạt nặng. Với ví dụ $A > B > C$ trong ảnh, một xếp hạng 3 phản hồi cho ra 3 cặp so sánh (A,B), (A,C), (B,C).

4. Mục tiêu của PPO và hệ số KL

$$\max_{y \sim \pi_\theta} E [R(y)] - \beta \cdot \text{KL}(\pi_\theta \parallel \pi_{\text{ref}})$$

Số hạng thứ nhất: sinh câu trả lời được chấm cao. Số hạng thứ hai: đừng đi quá xa model SFT ban đầu. Nếu không có ràng buộc KL, model sẽ tìm ra "mánh" để lừa reward model — **reward hacking** — ví dụ viết dài lê thê vì reward model vô tình thích câu dài, hoặc lặp lại cụm từ mà reward model đánh giá cao.

5. Các phương pháp thay thế hiện đại

- **DPO** (Direct Preference Optimization, 2023): bỏ hẳn reward model và vòng RL, tối ưu trực tiếp trên cặp ưa thích/bị loại bằng một hàm loss dạng phân loại. Đơn giản, ổn định, được dùng rộng rãi cho model mở.
- **RLAIF / Constitutional AI**: dùng chính một model AI, được hướng dẫn bởi một bộ nguyên tắc, để tạo phản hồi so sánh thay cho (một phần) con người — giảm chi phí gán nhãn.

6. Đọc bảng SFT vs RLHF

Điểm mấu chốt nằm ở hàng "Tín hiệu huấn luyện": SFT học từ *đáp án mẫu* (ground truth), RLHF học từ *sở thích tương đối*. Hàng "Output": SFT có thể "cứng nhắc" vì chỉ bắt chước; RLHF linh hoạt hơn nhưng có rủi ro mới (reward hacking, nịnh người dùng — sycophancy).

7. Ý nghĩa với người xây ứng dụng

Bạn hiếm khi tự làm RLHF, nhưng hiểu nó giúp giải thích các hành vi của chat model: vì sao model hay từ chối, vì sao đôi khi đồng ý với người dùng dù người dùng sai, vì sao model trả lời dài. Khi viết system prompt cho chatbot RAG, bạn đang "đề" lên các thói quen đã được hình thành từ giai đoạn alignment.

CHƯƠNG 8 · ẢNH GỐC

8. Nền tảng về RAG



8. NỀN TẢNG VỀ RETRIEVAL-AUGMENTED GENERATION (RAG)

RAG & LLMs
SERIES VIẾT TAY
v1.0

Đưa kiến thức của riêng bạn vào LLM

1. RAG LÀ GÌ?

RAG (Retrieval-Augmented Generation) là một kỹ thuật kết hợp sức mạnh của LLM với các nguồn kiến thức bên ngoài.

Ý tưởng: Thay vì chỉ dựa vào dữ liệu huấn luyện của LLM, ta truy xuất thông tin liên quan từ một knowledge base và đưa nó cho model làm ngữ cảnh.

Mục tiêu:

- Đưa ra câu trả lời chính xác, cập nhật và chuyên sâu theo lĩnh vực, ít hallucination hơn.

2. VÌ SAO CẦN RAG?

Vấn đề của LLM (khi dùng một mình)

- ❌ Kiến thức bị giới hạn ở mốc training cut-off
- ❌ Có thể đưa ra thông tin lỗi thời
- ❌ Hallucination với dữ liệu cụ thể / riêng tư
- ❌ Không thể truy cập tài liệu hoặc database riêng của bạn

RAG giúp gì

- ✅ Truy cập dữ liệu mới nhất hoặc dữ liệu riêng tư
- ✅ Phản hồi chính xác hơn và có căn cứ (grounded)
- ✅ Giảm hallucination
- ✅ Bạn kiểm soát nguồn kiến thức



3. RAG vs LLM (Không có RAG)

LLM (Không có RAG)

- ⚡ Trả lời chỉ từ kiến thức nội tại
- ⚡ Có thể lỗi thời
- ⚡ Khả năng sai / bịa câu trả lời cao hơn
- ⚡ Không truy cập được dữ liệu riêng của bạn

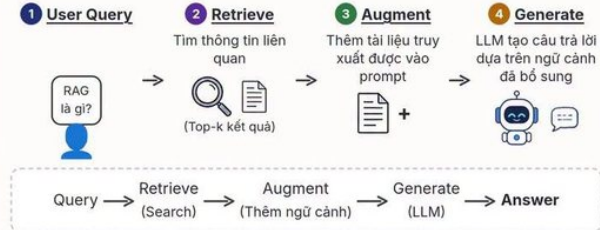


RAG (Có RAG)

- ✅ Trả lời dựa trên ngữ cảnh liên quan được truy xuất
- ✅ Thông tin cập nhật
- ✅ Chính xác hơn và có căn cứ
- ✅ Dùng dữ liệu riêng / chuyên ngành của bạn



4. RAG PIPELINE (TỔNG QUAN)

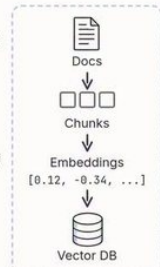


5. CÁC THÀNH PHẦN CỐT LÕI

Knowledge Source	Nơi lưu dữ liệu của bạn (PDF, Docs, DB, Website, API, Confluence, v.v.)
Retriever	Tìm kiếm trong knowledge base và trả về các chunk liên quan nhất.
Augmentor	Kết hợp ngữ cảnh truy xuất được với truy vấn của người dùng để tạo prompt nâng cao.
LLM (Generator)	Tạo câu trả lời cuối cùng dựa trên prompt đã bổ sung.
Answer	Phản hồi cuối cùng hiển thị cho người dùng.

6. CÁCH HOẠT ĐỘNG (TỪNG BƯỚC)

- Ingestion:** Tải tài liệu từ nhiều nguồn khác nhau.
- Chunking:** Chia tài liệu thành các chunk nhỏ hơn, có ý nghĩa.
- Embedding:** Chuyển các chunk thành biểu diễn vector.
- Indexing:** Lưu embeddings vào Vector Database.
- Query Embedding:** Chuyển truy vấn của người dùng thành vector.
- Retrieval:** Tìm top-k chunk liên quan từ vector DB.
- Augmentation:** Thêm các chunk này vào prompt.
- Generation:** LLM tạo câu trả lời cuối cùng.



7. CÁC LOẠI RAG

- ⚡ **Naive RAG**
Truy xuất top-k chunk và đưa thẳng vào LLM.
- ⚡ **Advanced RAG**
Bao gồm reranking, query rewriting, metadata filtering.
- ⚡ **Hybrid RAG (Sparse + Dense)**
Kết hợp keyword search (BM25) + vector search.
- ⚡ **Multi-Modal RAG**
Truy xuất từ văn bản, bảng, hình ảnh, code, v.v.

8. LƯỚING VÍ DỤ (HỖ TRỢ E-COMMERCE)

User: "Chính sách đổi trả cho sản phẩm mua khi giảm giá là gì?"

Retrieve: Lấy các phần chính sách liên quan từ tài liệu

Augment: Thêm chúng vào prompt

Generate: LLM tạo câu trả lời

Answer: "Sản phẩm giảm giá có thể được đổi trả trong vòng 7 ngày nếu còn nguyên tình trạng ban đầu..."

9. LỢI ÍCH

- ✅ Kiến thức luôn cập nhật
- ✅ Câu trả lời chuyên sâu theo lĩnh vực
- ✅ Chính xác & đáng tin cậy hơn
- ✅ Giảm hallucination
- ✅ Kiểm soát được kiến thức
- ✅ Phù hợp hơn cho doanh nghiệp và dữ liệu riêng tư



CHƯƠNG 8 · GIẢI THÍCH CHI TIẾT

Nền tảng về RAG

1. Ý tưởng cốt lõi: thi "mở sách"

LLM không có RAG giống một sinh viên thi *đóng sách*: trả lời bằng trí nhớ, nhớ sai thì bịa. RAG biến nó thành bài thi *mở sách*: trước khi trả lời, hệ thống lật đúng trang tài liệu liên quan và đặt trước mặt model. Thuật ngữ được giới thiệu trong bài báo của Lewis và cộng sự (Facebook AI, 2020).

2. Bốn vấn đề RAG giải quyết

Vấn đề của LLM	RAG xử lý thế nào
Kiến thức dừng ở mốc training cut-off	Tài liệu mới chỉ cần index thêm
Không biết dữ liệu riêng	Đưa dữ liệu riêng vào ngữ cảnh lúc hỏi
Hallucination	Buộc model trả lời dựa trên đoạn văn có thật
Không trích dẫn được	Mỗi chunk mang metadata nguồn → trích dẫn

Một lợi ích ít được nhắc: **phân quyền**. Có thể lọc tài liệu theo quyền của người hỏi trước khi đưa vào model — điều không làm được nếu kiến thức đã "nướng" vào trọng số bằng fine-tuning.

3. Hai pha của RAG

Ảnh "Cách hoạt động (từng bước)" liệt kê 8 bước; chúng chia thành hai pha chạy ở hai thời điểm khác nhau:

Pha	Bước	Khi nào chạy
Indexing (offline)	1 Ingestion → 2 Chunking → 3 Embedding → 4 Indexing	Khi có tài liệu mới/thay đổi
Querying (online)	5 Query embedding → 6 Retrieval → 7 Augmentation → 8 Generation	Mỗi câu hỏi của người dùng

Điều kiện bắt buộc: **query và chunk phải được embed bằng cùng một embedding model**. Đổi model embedding = phải index lại toàn bộ.

4. Năm thành phần

- **Knowledge source**: nơi chứa dữ liệu gốc (PDF, DOCX, web, database, API).
- **Retriever**: tìm top-k chunk liên quan (vector search, BM25, hoặc kết hợp).
- **Augmentor**: ghép chunk + câu hỏi vào prompt template.
- **Generator**: LLM sinh câu trả lời.
- **Answer**: phản hồi cuối, lý tưởng kèm trích dẫn.

5. Ví dụ chạy tay với chatbot quy chế

Câu hỏi: "Sinh viên bị cảnh báo học tập khi nào?"

- **Retrieve**: vector câu hỏi gần nhất với chunk "Điều 12. Cảnh báo kết quả học tập..." (score 0,83), chunk "Điều 13. Buộc thôi học..." (0,71), chunk "Điều 10. Đánh giá học phần..." (0,58).

- **Augment:** chèn 3 chunk này (kèm nhãn nguồn) vào template ở Chương 4.
- **Generate:** model trả lời dựa trên Điều 12, ghi "(Nguồn: Quy chế đào tạo, Điều 12)". Nếu không có chunk nào phù hợp, model được dặn trả lời "chưa tìm thấy quy định".

6. Bốn cấp độ RAG

- **Naive RAG:** chunk cố định + vector search + top-k thẳng vào prompt. Làm được trong một buổi; đủ cho bản demo.
- **Advanced RAG:** thêm xử lý trước truy xuất (viết lại câu hỏi), sau truy xuất (rerank, lọc metadata, nén ngữ cảnh).
- **Hybrid RAG:** kết hợp BM25 (khớp từ khóa chính xác — rất quan trọng với mã học phần, số hiệu điều khoản) và vector search (khớp ý nghĩa).
- **Multi-modal RAG:** truy xuất cả bảng biểu, hình ảnh.

Lời khuyên: bắt đầu bằng Naive RAG, **đo lường** (Chương 14), rồi mới thêm từng kỹ thuật nâng cao khi số liệu cho thấy cần. Thêm kỹ thuật mà không đo thì không biết nó giúp hay hại.

7. Hạn chế của RAG (ảnh chỉ nêu lợi ích)

- Chất lượng phụ thuộc retrieval: **tìm sai chunk thì trả lời sai** — "garbage in, garbage out".
- Câu hỏi cần tổng hợp toàn bộ kho ("tóm tắt mọi thay đổi trong quy chế năm nay") khó với top-k.
- Thêm độ trễ (embedding + search + rerank) và thêm thành phần cần vận hành.
- Rủi ro prompt injection gián tiếp: tài liệu được index có thể chứa chỉ dẫn độc hại.

CHƯƠNG 9 · ẢNH GỐC

9. Document Loading & Chunking

RAG & LLMs
SERIES VIẾT TAY
v1.0



9. DOCUMENT LOADING & CHUNKING

Dữ liệu tốt vào → Chunk tốt ra → RAG tốt hơn

1. DOCUMENT LOADING LÀ GÌ?

Document Loading là quá trình nhập dữ liệu từ nhiều nguồn khác nhau vào RAG pipeline theo một định dạng có cấu trúc để có thể xử lý được.

- ✓ Hỗ trợ nhiều loại file và nguồn
- ✓ Chuyển dữ liệu thô sang định dạng chuẩn
- ✓ Chuẩn bị dữ liệu cho chunking và embedding

Mục tiêu:
Đưa toàn bộ kiến thức của bạn vào hệ thống một cách sạch sẽ và dễ sử dụng.

2. CÁC NGUỒN DOCUMENT LOADING

File PDF
(Báo cáo, bài báo, tài liệu hướng dẫn)

File Word / Docx

File Text (.txt)

Trang web (URL)

Database (SQL, NoSQL)

Email (.eml)

Cloud Storage (Drive, S3, OneDrive)

Công cụ / thư viện phổ biến: LangChain Document Loaders, LlamaIndex Readers, Unstructured, PyPDF, BeautifulSoup, v.v.

3. DOCUMENT LOADING PIPELINE (TỔNG QUAN)

```

Source (Nơi chứa dữ liệu) → Loader (Đọc & trích xuất) → Parser (Trích xuất văn bản) → Document Objects (Định dạng chuẩn)

```

Document Object (Ví dụ)

```

page_content: "Đây là một đoạn văn bản mẫu..."
metadata: {
  "source": "example.pdf",
  "page": 1,
  "author": "An"
}

```

Metadata cực kỳ quan trọng cho việc lọc (filtering) và truy vết (traceability)!

4. VÌ SAO CẦN CHUNKING?

Vấn đề

- ✗ Tài liệu quá lớn so với context window của LLM
- ✗ Thông tin không liên quan làm tăng nhiễu
- ✗ Khó truy xuất chính xác thông tin
- ✗ Lãng phí token

Chunking giúp

- ✓ Chia tài liệu lớn thành các phần nhỏ, có ý nghĩa
- ✓ Cải thiện độ chính xác khi truy xuất
- ✓ Vừa với context window
- ✓ Semantic search tốt hơn

Quy tắc vàng: Chunk thông minh, không chỉ là chunk nhỏ.

5. CÁC CHIẾN LƯỢC CHUNKING

Chiến lược	Cách hoạt động	Phù hợp cho
Fixed Size	Chia văn bản thành các chunk có kích thước cố định (ví dụ: 500 token)	Mục đích chung
Recursive Character Text Splitter	Chia theo đoạn văn → câu → từ nếu lớn hơn chunk size	Phổ biến nhất & hiệu quả
Semantic Chunking	Nhóm văn bản dựa trên độ tương đồng ngữ nghĩa (dùng embeddings)	Giữ ngữ cảnh tốt hơn
Document Structure Based	Dùng heading, section, tiêu đề để chia	Tài liệu kỹ thuật, Markdown, HTML

6. THAM SỐ CHUNKING (HÃY TÍNH CHÍNH!)

- Chunk Size**
Số ký tự/token trong mỗi chunk.
Thông thường: 300 - 1000 token
- Chunk Overlap**
Số token chồng lấn giữa các chunk liên tiếp để giữ ngữ cảnh.
Thông thường: 50 - 200 token
- Separator**
Các ký tự dùng để chia (\\n, ., !, ? v.v.)
- Length Function**
Hàm đo kích thước (len, tiktoken, v.v.)

Cấu hình ví dụ

```

chunk_size = 800
chunk_overlap = 100
separators = ["\\n\\n", "\\n", ".", "!", "?"]
length_function = tiktoken_len

```

Mẹo:
Chunk nhỏ hơn → truy xuất chính xác hơn nhưng nhiều embedding hơn.
Chunk lớn hơn → nhiều ngữ cảnh hơn nhưng truy xuất kém chính xác hơn.

7. VÍ DỤ CHUNKING

Tài liệu gốc

RAG kết hợp retrieval và generation. Đầu tiên nó truy xuất các tài liệu liên quan từ một knowledge base và sau đó dùng chúng làm ngữ cảnh để tạo ra câu trả lời chính xác. Điều này giúp giảm hallucination và cải thiện độ chính xác về mặt dữ kiện.

Sau khi Chunking (Size = 50 từ, Overlap = 10 từ)

Chunk 1: RAG kết hợp retrieval và generation. Đầu tiên nó truy xuất các tài liệu liên quan từ một...

Chunk 2: ...knowledge base và sau đó dùng chúng làm ngữ cảnh để tạo ra câu trả lời chính xác. Điều này giúp...

Chunk 3: ...giảm hallucination và cải thiện độ chính xác về mặt dữ kiện.

8. BEST PRACTICES

- ✓ Luôn làm sạch văn bản trước khi chunking.
- ✓ Loại bỏ boilerplate, header, footer, số trang
- ✓ Dùng recursive splitter để duy trì ngữ cảnh tốt nhất
- ✓ Giữ overlap để duy trì ngữ cảnh
- ✓ Lưu metadata cùng mỗi chunk (source, page, title, date, v.v.)
- ✓ Thử nghiệm với các chunk size khác nhau tùy use case
- ✓ Kiểm tra thủ công các chunk với tài liệu quan trọng

Không nên:

- ✗ Chunk quá nhỏ (mất ngữ cảnh)
- ✗ Chunk quá lớn (thông tin không liên quan)
- ✗ Không có overlap
- ✗ Bỏ qua metadata

9. CÔNG CỤ CÓ THỂ DÙNG

LangChain: PyPDFLoader, WebBaseLoader, TextLoader, CSVLoader, v.v.

10. MINI CHECKLIST

- ✓ Mình đã load đúng dữ liệu chưa?
- ✓ Dữ liệu đã sạch và có cấu trúc chưa?

CHƯƠNG 9 · GIẢI THÍCH CHI TIẾT

Document Loading & Chunking

1. Loading: bước bị đánh giá thấp nhất

Phần lớn lỗi RAG trong thực tế bắt nguồn từ dữ liệu đầu vào chứ không phải từ LLM. Một PDF trích xuất sai thứ tự cột, bảng bị vỡ thành chuỗi số vô nghĩa, header/footer lặp lại trên mọi trang — tất cả đều đi thẳng vào vector DB và làm hỏng kết quả tìm kiếm.

Nguồn	Công cụ gợi ý	Cạm bẫy thường gặp
PDF có text	PyMuPDF, pdfplumber, PyPDF	Hai cột bị trộn dòng; bảng vỡ
PDF scan	OCR (Tesseract, PaddleOCR) hoặc model đọc ảnh	Lỗi nhận dạng dấu tiếng Việt
DOCX	python-docx, Unstructured	Mất cấu trúc heading nếu đọc thô
HTML/web b	BeautifulSoup, trafilatura	Menu, quảng cáo lẫn vào nội dung
Database/API	Truy vấn trực tiếp	Nên chuyển thành văn bản có ngữ cảnh ("Học phần X có 3 tín chỉ")

2. Document object và metadata

Mỗi đoạn sau khi load nên là một object gồm **page_content** và **metadata**. Metadata tốt cho phép: lọc (chỉ tìm trong văn bản còn hiệu lực), trích dẫn (Điều mấy, trang mấy), và gỡ lỗi (chunk này từ file nào). Ví dụ cho tài liệu quy chế:

```
{
  "page_content": "Điều 12. Chế độ báo cáo kết quả học tập ...",
  "metadata": {
    "source": "quy_che_dao_tao_2024.pdf",
    "chuong": "III", "dieu": 12, "page": 9,
    "hieu_luc_tu": "2024-09-01", "doi_tuong": "chinh_quy"
  }
}
```

3. Vì sao phải chunk

- Tài liệu dài hơn context window.
- Một embedding cho cả tài liệu 50 trang là "trung bình" của quá nhiều chủ đề → vector mờ nhạt, tìm kiếm kém.
- Chỉ đưa phần liên quan vào prompt → ít nhiễu, ít tốn token.

Nguyên tắc vàng trong ảnh — "chunk thông minh, không chỉ chunk nhỏ" — có nghĩa: **mỗi chunk nên tự đứng được**, một người đọc riêng chunk đó vẫn hiểu nó nói về gì.

4. Bốn chiến lược chunking

- **Fixed size**: cắt mỗi N token. Đơn giản, nhưng dễ cắt giữa câu.
- **Recursive character splitter**: thử tách theo "\n\n" (đoạn) trước; đoạn nào vẫn quá dài thì tách theo "\n", rồi ".", rồi " ". Giữ ranh giới tự nhiên nhiều nhất có thể — lựa chọn mặc định tốt.

- **Semantic chunking**: embed từng câu, cắt ở chỗ độ tương đồng giữa hai câu liền kề giảm mạnh (đổi chủ đề). Tốn thêm chi phí embedding.
- **Structure-based**: cắt theo heading/section. **Rất phù hợp văn bản pháp quy tiếng Việt**: mỗi "Điều" là một đơn vị ý nghĩa tự nhiên; Điều quá dài thì tách theo Khoản, và gắn tiêu đề Điều vào đầu mỗi mảnh.

5. Chunk size và overlap — đánh đổi

Chunk nhỏ (100-300 token)	Chunk lớn (800-1500 token)
Embedding sắc nét, tìm chính xác	Embedding mờ hơn
Dễ thiếu ngữ cảnh để trả lời	Đủ ngữ cảnh
Cần top-k lớn hơn	top-k nhỏ là đủ

Overlap (thường 10-20% chunk size) để một ý nằm ở ranh giới không bị cắt đôi mất nghĩa. Ví dụ tính: tài liệu 10.000 token, chunk 800, overlap 100 → bước nhảy 700 → khoảng $\lceil (10.000 - 800) / 700 \rceil + 1 = 15$ chunk.

Một kỹ thuật kết hợp được ưa chuộng là **small-to-big** (parent document retrieval): tìm kiếm trên chunk nhỏ để chính xác, nhưng khi đưa vào prompt thì đưa cả đoạn cha lớn hơn để đủ ngữ cảnh.

6. Code mẫu

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
import tiktoken
enc = tiktoken.get_encoding("cll00k_base")
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500, chunk_overlap=80,
    length_function=lambda t: len(enc.encode(t)),
    separators=["\nĐiều ", "\n\n", "\n", ". ", " "],
)
chunks = splitter.split_text(van_ban_quy_che)
```

Chú ý separator đầu tiên "\nĐiều ": splitter sẽ ưu tiên cắt ở đầu mỗi Điều — một cách đơn giản để có chunking theo cấu trúc.

7. Checklist trước khi embed

- Mở ngẫu nhiên 20 chunk và **đọc bằng mắt**. Chunk có tự hiểu được không? Có rác (số trang, header) không?
- Kiểm tra bảng biểu: bảng nên được chuyển thành Markdown hoặc thành câu ("Học kỳ chính: tối đa 25 tín chỉ").
- Chuẩn hóa Unicode tiếng Việt (NFC) — cùng một chữ "ệ" có thể được mã hóa theo hai cách, khiến BM25 không khớp.
- Lưu metadata đủ để trích dẫn.

CHƯƠNG 10 · ẢNH GỐC

10. Embeddings & Vector Databases



10. EMBEDDINGS & VECTOR DATABASES

RAG & LLMs
SERIES VIẾT TAY
v1.0

Biến văn bản thành vector. Lưu trữ thông minh. Tìm kiếm tốt hơn. 🤖

1. EMBEDDINGS LÀ GÌ?

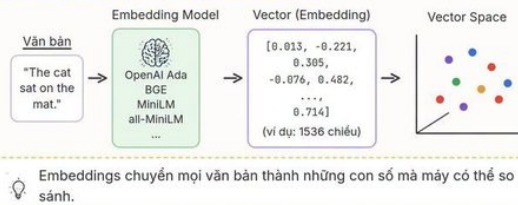
Embeddings là các biểu diễn vector dạng số của văn bản (hoặc dữ liệu khác) nắm bắt ý nghĩa trong một không gian nhiều chiều.

- ✓ Nghĩa tương tự → vector gần nhau
- ✓ Nghĩa khác nhau → vector xa nhau
- ✓ Hỗ trợ semantic search, clustering, retrieval và nhiều hơn nữa

Ví dụ

"king" - "man" + "woman" ≈ "queen"
(Embeddings nắm bắt được các mối quan hệ! 🧠)

2. EMBEDDINGS HOẠT ĐỘNG THE NÀO



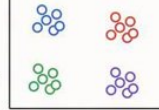
Embeddings chuyển mọi văn bản thành những con số mà máy có thể so sánh.

3. ĐẶC TÍNH CỦA EMBEDDINGS TỐT

- 🔥 **Semantic Meaning:** Các văn bản tương tự nằm gần nhau trong không gian vector
- 🔥 **Generalization:** Hoạt động tốt với dữ liệu chưa từng thấy
- 🔥 **Contextual:** Nắm bắt ý nghĩa trong ngữ cảnh
- 🔥 **Robustness:** Xử lý được nhiễu và các biến thể
- 🔥 **Compact:** Lượng thông tin lớn trong vector có kích thước cố định

4. TRỰC QUAN HÓA EMBEDDINGS (VÍ DỤ 2D)

Trong không gian nhiều chiều (768, 1024, 1536...), các mục tương tự nằm gần nhau.



- 🔵 Động vật (mèo, chó, ngựa)
- 🟢 Trái cây (táo, xoài, chuối)
- 🟡 Phương tiện (ô tô, xe đạp, tàu hỏa)
- 🔴 Quốc gia (Ấn Độ, Mỹ, Nhật Bản)
- 🟠 ...

5. CÁC EMBEDDING MODEL PHỔ BIẾN

Model	Số chiều	Điểm mạnh	Use Cases
OpenAI text-embedding-3-small	1536	Chất lượng cao, hiệu năng mạnh	Mục đích chung, RAG, search
BAAI/bge-small-en-v1.5	384	Chất lượng tốt, nhẹ, open source	RAG, semantic search
sentence-transformers/all-MiniLM-L6-v2	384	Nhanh, nhẹ, open source	Prototyping, ứng dụng nhỏ
intfloat/multilingual-e5-small	384	Hỗ trợ đa ngôn ngữ	RAG đa ngôn ngữ
Cohere embed-english-v3.0	1024	Mạnh cho retrieval	Enterprise search

Chọn model dựa trên chất lượng, kích thước, độ trễ (latency), chi phí & khả năng hỗ trợ ngôn ngữ.

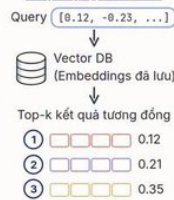
6. VECTOR DATABASE - LÀ GÌ & VÌ SAO?

Vector Database lưu trữ embeddings và cho phép tìm kiếm tương đồng (similarity search) nhanh.

Vì sao cần chúng?

- ✓ Lưu trữ hàng triệu vector
- ✓ Thực hiện similarity search (nearest neighbors)
- ✓ Mở rộng tốt & tối ưu cho dữ liệu nhiều chiều
- ✓ Làm nền tảng cho RAG và các ứng dụng semantic

Hoạt động thế nào?



7. VECTOR SEARCH (TÌM KIẾM TƯƠNG ĐỒNG)

Cho một query vector q và các document vector $d_1, d_2, \dots, d_n$, tìm các vector tương đồng nhất.

Ví dụ Cosine Similarity

Query (q) [0.1 -0.2 0.3 0.4 ...]

Document Vectors

d_1 [0.1 -0.1 0.2 0.3 ...] 0.92

d_2 [-0.2 0.1 -0.3 0.2 ...] 0.21

d_3 [0.3 0.4 -0.1 0.2 ...] 0.15

Kết quả top → d_1 (tương đồng nhất) 🏆

Các metric tương đồng phổ biến

★ Cosine Similarity (phổ biến nhất)

$$\cos(\theta) = \frac{A \cdot B}{|A| |B|}$$

★ Dot Product

$$A \cdot B = \sum_i A_i B_i$$

★ Euclidean Distance

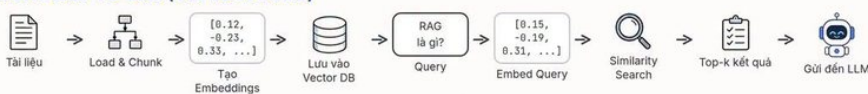
$$d(A, B) = \sqrt{\sum_i (A_i - B_i)^2}$$

8. CÁC VECTOR DATABASE PHỔ BIẾN

Vector DB	Loại	Điểm mạnh	Phù hợp cho
Pinecone	Managed (Cloud)	Dễ dùng, mở rộng tốt, hiệu năng cao	Production RAG, enterprise
Weaviate	Open Source (Self/Cloud)	Hybrid search, filtering, GraphQL, modules	Semantic search, ứng dụng AI
Qdrant	Open Source (Self/Cloud)	Nhanh, nhẹ, payload filtering	RAG, recommendation, ứng dụng real-time
Milvus	Open Source (Self/Cloud)	Mở rộng cao, hỗ trợ multi-modal	Quy mô lớn, vector workload phức tạp
Chroma	Open Source (Local/Cloud)	Đơn giản, thân thiện với developer, xây dựng cho ứng dụng LLM	Prototyping, ứng dụng RAG nhỏ

★ Tất cả đều hỗ trợ similarity search trên vector nhiều chiều.

9. LƯỚI END-TO-END (VỚI VECTOR DB)



Dữ liệu tốt vào → truy xuất tốt hơn → câu trả lời tốt hơn 🧡

CHƯƠNG 10 · GIẢI THÍCH CHI TIẾT

Embeddings & Vector Databases

1. Embedding model trong RAG

Embedding model nhận một đoạn văn và trả về một vector cố định (ví dụ 384, 768, 1024 hay 1536 chiều) sao cho các đoạn có ý nghĩa gần nhau thì vector gần nhau. Chúng thường là encoder kiểu BERT được huấn luyện bằng **contrastive learning**: kéo cặp (câu hỏi, đoạn trả lời đúng) lại gần, đẩy các cặp sai ra xa.

2. Ba thước đo tương đồng — tính tay

Cho $a = [1, 2, 2]$ và $b = [2, 1, 2]$.

- Dot product: $1 \cdot 2 + 2 \cdot 1 + 2 \cdot 2 = 8$.
- Độ dài: $|a| = \sqrt{1+4+4} = 3$; $|b| = 3$.
- Cosine: $8 / (3 \times 3) \approx 0,889$.
- Euclid: $\sqrt{((1-2)^2 + (2-1)^2 + (2-2)^2)} = \sqrt{2} \approx 1,414$.

$$\cos(\theta) = (A \cdot B) / (|A| \cdot |B|) \quad d(A, B) = \sqrt{(\sum (A_i - B_i)^2)}$$

Mối liên hệ quan trọng: nếu mọi vector đã được **chuẩn hóa về độ dài 1** (đa số embedding model làm vậy hoặc nên làm), thì cosine = dot product, và $\|a - b\|^2 = 2 - 2 \cdot \cos$. Ba thước đo cho cùng một thứ tự xếp hạng. Nhưng phải dùng đúng metric mà model được huấn luyện.

3. Chọn embedding model cho tiếng Việt

Bảng trong ảnh phần lớn là model tiếng Anh (bge-small-en, all-MiniLM). Với tài liệu tiếng Việt, cần model **đa ngôn ngữ**:

Model	Chiều	Ghi chú
BAAI/bge-m3	1024	Đa ngôn ngữ mạnh, hỗ trợ dense + sparse, văn bản dài (8K token)
intfloat/multilingual-e5 (small/base/large)	384/768/1024	Cần tiền tố "query: " và "passage: "
Model embedding qua API (OpenAI, Cohere, Google...)	768–3072	Không cần GPU, tính phí theo token
Model tiếng Việt chuyên biệt (cộng đồng)	768	Cần tự đánh giá trên dữ liệu của bạn

Cách chọn đúng: lấy 50 câu hỏi thật + chunk đáp án đúng, đo Recall@5 cho từng model ứng viên, chọn model tốt nhất trên **dữ liệu của bạn** — bảng xếp hạng chung (như MTEB) chỉ là điểm khởi đầu.

4. Vector database làm gì mà database thường không làm được

Tìm chính xác (brute force) nghĩa là tính khoảng cách tới **mọi** vector: 1 triệu vector $\times$ 1024 chiều $\approx$ 1 tỷ phép nhân cho mỗi truy vấn. Vector DB dùng chỉ mục **ANN** (Approximate Nearest Neighbor) để đổi một chút độ chính xác lấy tốc độ gấp hàng trăm lần:

- **HNSW**: đồ thị nhiều tầng, tầng trên thưa để "nhảy xa", tầng dưới dày để "tinh chỉnh". Phổ biến nhất, nhanh, tốn RAM.

- **IVF**: phân cụm vector, chỉ tìm trong vài cụm gần nhất.
- **PQ** (Product Quantization): nén vector để tiết kiệm bộ nhớ.

5. Chọn vector DB

Quy mô / nhu cầu	Gợi ý
Học, prototype, <100K chunk	Chroma, FAISS (thư viện, không phải server)
Đã có PostgreSQL, muốn ít thành phần	pgvector
Tự host production, cần lọc metadata mạnh	Qdrant, Weaviate, Milvus
Không muốn vận hành	Pinecone hoặc dịch vụ managed khác

Với một chatbot quy chế (vài nghìn chunk), gần như mọi lựa chọn đều đủ nhanh; hãy ưu tiên cái dễ vận hành và hỗ trợ **lọc metadata + hybrid search**.

6. Code tối giản

```
from sentence_transformers import SentenceTransformer
import chromadb
model = SentenceTransformer("BAAI/bge-m3")
db = chromadb.PersistentClient("./db").get_or_create_collection(
    "quy_che", metadata={"hnsw:space": "cosine"})
vecs = model.encode(chunks, normalize_embeddings=True)
db.add(ids=[f"c{i}" for i in range(len(chunks))],
      documents=chunks, embeddings=vecs.tolist(), metadatas=metas)
q = model.encode(["Khi nào bị c[]nh báo học tập?"],
                 normalize_embeddings=True)
res = db.query(query_embeddings=q.tolist(), n_results=5)
```

Đính chính nhỏ: phép "king – man + woman $\approx$ queen" được chứng minh với word2vec (embedding từ đơn); embedding câu hiện đại không nhất thiết có tính chất số học gọn gàng như vậy. Trong ảnh "Vector Search", điểm 0,12 / 0,21 / 0,35 ở mục "Top-k" có vẻ là *khoảng cách* (càng nhỏ càng gần), còn 0,92 / 0,21 / 0,15 ở ví dụ cosine là *độ tương đồng* (càng lớn càng gần) — luôn kiểm tra thư viện của bạn trả về loại nào.

CHƯƠNG 11 · ẢNH GỐC

11. Retrieval & Reranking



11. RETRIEVAL & RERANKING

RAG & LLMs
SERIES VIẾT TAY
v1.0

Tìm đúng. Xếp hạng tốt hơn. Sinh câu trả lời thông minh hơn. 🎯

1. RETRIEVAL & RERANKING LÀ GÌ?

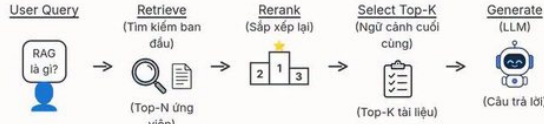
Retrieval tìm một tập các tài liệu liên quan (ứng viên - candidates) từ knowledge base.

Reranking sắp xếp lại các ứng viên đó bằng một model mạnh hơn để đưa các tài liệu liên quan nhất lên đầu.

Mục tiêu:

- Đảm bảo ngữ cảnh liên quan nhất đến được LLM để có câu trả lời chính xác và đáng tin cậy.

2. VỊ TRÍ TRONG RAG PIPELINE



Retrieval tốt hơn + reranking tốt hơn = câu trả lời tốt hơn

3. RETRIEVAL - CÁCH HOẠT ĐỘNG

- Query Embedding:** Chuyển query thành vector.
- Vector Search:** Tìm các vector tương đồng trong Vector DB.
- Top-N Candidates:** Trả về N chunk tương đồng nhất.
- Pass to Reranker:** Gửi các ứng viên để chấm điểm lại.
- Select Top-K:** Dùng K chunk hàng đầu sau reranking cho LLM.

**Các phương pháp Retrieval phổ biến**

- Dense Retrieval (Vector Search)
- Sparse Retrieval (BM25 / Keyword Search)
- Hybrid Retrieval (Dense + Sparse)

4. RERANKING - VÌ SAO & THẾ NÀO?

Vì sao cần Rerank?

- Retrieval ban đầu có thể bỏ sót tài liệu tốt nhất.
- Độ tương đồng ngữ nghĩa $\neq$ độ liên quan thực sự.
- Reranking cải thiện độ chính xác (precision) nhờ hiểu ngữ cảnh sâu hơn.

Reranking hoạt động thế nào?

- Nhận query + tài liệu ứng viên.
- Dùng cross-encoder (hoặc LLM) để chấm điểm mức độ liên quan.
- Trả về một relevance score.
- Sắp xếp các ứng viên theo điểm (cao $\rightarrow$ thấp).

Ví dụ (Trước & Sau Reranking)

Trước Reranking (theo similarity)	Reranker (Cross-Encoder)	Sau Reranking (theo relevance)
1. Doc A (score: 0.78) 2. Doc B (score: 0.72) 3. Doc C (score: 0.71) 4. Doc D (score: 0.70)		1. Doc C (relevance: 0.92) 2. Doc A (relevance: 0.89) 3. Doc D (relevance: 0.76) 4. Doc B (relevance: 0.58)

5. RETRIEVAL vs RERANKING

Khía cạnh	Retrieval	Reranking
Mục đích	Tìm các ứng viên liên quan	Sắp xếp lại ứng viên theo độ liên quan thực sự
Input	Query	Query + các ứng viên đã truy xuất
Phương pháp	Bi-encoder / Dense / Sparse / Hybrid	Cross-encoder / LLM Reranker
Output	Top-N tài liệu tương đồng	Top-K tài liệu liên quan nhất
Tốc độ	Nhanh ⚡	Chậm hơn 🐢
Chi phí	Thấp 💰	Cao hơn 💸
Vị trí trong RAG	Bước đầu tiên	Sau retrieval, trước LLM

6. CÁC RERANKER MODEL PHỔ BIẾN

Model / Cách tiếp cận	Loại	Điểm mạnh	Use Cases
Cross-Encoder (ví dụ: bge-reranker, hkunlp/instructor-xl)	Transformer (Cross-Encoder)	Độ chính xác cao, hiểu ngữ cảnh sâu	RAG tổng quát, Q&A, search
LLM as Reranker (GPT-4o, Claude, Llama 3, v.v.)	Chấm điểm bằng LLM	Tốt nhất cho suy luận phức tạp, mức độ liên quan tinh tế	Lĩnh vực cần độ chính xác cao, ngữ cảnh dài
Bi-Encoder Score (ví dụ: Sentence-BERT)	Cosine Similarity (Nhanh)	Rất nhanh, mở rộng tốt	Khi tốc độ là quan trọng

★ Cross-encoder reranker cho chất lượng tốt nhất nhưng chậm hơn và tốn kém hơn.

7. CHẤM ĐIỂM TRONG RERANKING

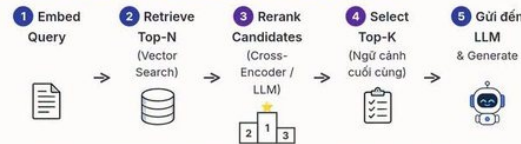
Reranker nhận vào:

- Query (Q)
- Candidate Document (D)
- Trả về Relevance Score (0 đến 1 hoặc thang điểm bất kỳ)

Các yếu tố được xét

- Mức độ liên quan ngữ nghĩa
- Hiểu ngữ cảnh
- Tầm quan trọng của từ khóa
- Vị trí & mức độ nhấn mạnh
- Độ mạch lạc & khớp ý định

8. LƯỖNG RETRIEVAL + RERANKING (CHI TIẾT)



9. BEST PRACTICES

- Retrieve nhiều (N lớn), rerank ít (K nhỏ).
- Dùng hybrid retrieval để có recall tốt hơn.

10. THÁCH THỨC

- Reranking chậm hơn và tốn kém hơn.
- Cross-encoder không mở rộng được với N quá lớn.

11. CÔNG CỤ & THƯ VIỆN PHỔ BIẾN

- Vector Search:** FAISS, Pinecone, Weaviate, Qdrant, Milvus
- Sparse Search:** BM25 (Elasticsearch), Lucene

CHƯƠNG 11 · GIẢI THÍCH CHI TIẾT

Retrieval & Reranking

1. Hai giai đoạn: lưới rộng rồi sàng mịn

Retrieval giống thả lưới rộng: nhanh, rẻ, bắt được nhiều cá (top-N = 20–100 ứng viên) nhưng lẫn cả rác. Reranking giống sàng mịn: chậm hơn, kỹ hơn, chọn ra top-K = 3–5 tốt nhất để đưa cho LLM. Nguyên tắc trong ảnh "retrieve nhiều (N lớn), rerank ít (K nhỏ)" chính là vậy.

2. Bi-encoder vs cross-encoder — điểm mấu chốt

Bi-encoder (retrieval)	Cross-encoder (reranking)
Mã hóa query và tài liệu riêng rẽ thành 2 vector	Đưa cặp [query, tài liệu] vào cùng một model
Vector tài liệu tính trước, lưu sẵn	Phải chạy model cho từng cặp lúc truy vấn
So sánh bằng cosine — cực nhanh	Attention giữa từng từ của query và tài liệu — chính xác hơn
Tìm được trong hàng triệu tài liệu	Chỉ khả thi với vài chục ứng viên

Vì sao cross-encoder chính xác hơn? Bi-encoder phải nén cả đoạn văn vào một vector *trước khi biết câu hỏi là gì*. Cross-encoder đọc câu hỏi và tài liệu cùng lúc nên nhận ra được những chi tiết như phủ định ("*không* được đăng ký") hay điều kiện cụ thể.

3. Sparse retrieval: BM25

BM25 chấm điểm dựa trên từ khóa trùng khớp:

$$\text{score}(D,Q) = \sum_{q \in Q} \text{IDF}(q) \cdot f(q,D) \cdot (k_1 + 1) / (f(q,D) + k_1 \cdot (1 - b + b \cdot |D|/\text{avgdl}))$$

- $f(q,D)$: số lần từ q xuất hiện trong D — nhưng có **bão hòa** (xuất hiện 10 lần không gấp 10 lần điểm), điều khiển bởi $k_1 \approx 1,2-2$.
- IDF: từ hiếm (như "HP0123", "cảnh báo") được trọng số cao; từ phổ biến ("sinh viên", "của") gần như không đóng góp.
- $b \approx 0,75$: phạt nhẹ tài liệu dài.

BM25 mạnh ở chỗ dense search yếu: mã học phần, số hiệu văn bản, tên riêng, thuật ngữ chính xác. Với tiếng Việt, nên tách từ (ví dụ "học_phần") hoặc dùng n-gram âm tiết để BM25 hoạt động tốt.

4. Hybrid search và RRF

Điểm BM25 (0 đến vài chục) và cosine (−1 đến 1) không cùng thang đo nên không cộng trực tiếp được. **Reciprocal Rank Fusion** chỉ dùng thứ hạng:

$$\text{RRF}(d) = \sum_{r \in \text{các bảng xếp hạng}} 1 / (k + \text{rank}_r(d)), \text{ k thường } = 60$$

Tính tay: tài liệu A hạng 1 trong BM25 và hạng 3 trong vector $\rightarrow 1/61 + 1/63 \approx 0,0164 + 0,0159 = 0,0323$. Tài liệu B hạng 2 và hạng 1 $\rightarrow 1/62 + 1/61 \approx 0,0161 + 0,0164 = 0,0325$. B xếp trên A một chút vì ổn định ở cả hai danh sách.

5. MMR — đa dạng hóa kết quả

Top-K thuần có thể trả về 5 chunk gần như trùng nhau (cùng một điều khoản xuất hiện ở hai phiên bản văn bản). **Maximal Marginal Relevance** chọn lần lượt chunk tối đa hóa:

$$\text{MMR} = \lambda \cdot \text{sim}(d, q) - (1 - \lambda) \cdot \max_{d' \in \text{đã chọn}} \text{sim}(d, d')$$

$\lambda = 1$ là top-K thuần; λ nhỏ hơn ưu tiên đa dạng. Giá trị 0,5–0,7 thường hợp lý.

6. Đọc ví dụ "Trước & Sau reranking"

Trước rerank: Doc A (0,78) đứng đầu vì nhiều từ giống câu hỏi. Sau rerank: Doc C (0,92) lên đầu vì cross-encoder nhận ra C mới thực sự trả lời câu hỏi. Đây chính là ý "độ tương đồng ngữ nghĩa $\neq$ độ liên quan thực sự": một đoạn có thể *nói về cùng chủ đề* mà không *trả lời câu hỏi*.

7. Code reranking

```
from sentence_transformers import CrossEncoder
reranker = CrossEncoder("BAAI/bge-reranker-v2-m3") # đa ngôn ngữ
cands = res["documents"][0] # 20 ứng viên từ vector DB
scores = reranker.predict([(cau_hoi, c) for c in cands])
top = [c for _, c in sorted(zip(scores, cands), reverse=True)][:4]
```

Đính chính: dòng "Bi-Encoder Score" trong bảng reranker thực chất không phải reranking — đó chính là điểm của bước retrieval. Ngoài ra, "hkunlp/instructor-xl" là một *embedding model* (bi-encoder), không phải cross-encoder. Các reranker phổ biến: bge-reranker (v2-m3 hỗ trợ tiếng Việt), Cohere Rerank, Jina Reranker, hoặc dùng LLM làm reranker.

CHƯƠNG 12 · ẢNH GỐC

12. Pipeline & Kiến trúc RAG



12. PIPELINE & KIẾN TRÚC RAG

RAG & LLMs
SERIES VIẾT TAY
v1.0



Đúng dữ liệu + Đúng ngữ cảnh = Đúng câu trả lời

1. RAG LÀ GÌ?

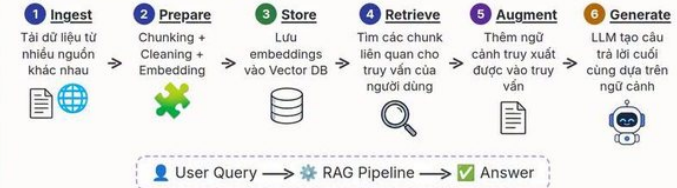
RAG (Retrieval-Augmented Generation) kết hợp sức mạnh của LLM với dữ liệu của riêng bạn.

- ✓ Truy xuất kiến thức liên quan từ các nguồn bên ngoài
- ✓ Bổ sung ngữ cảnh đó vào prompt
- ✓ LLM tạo câu trả lời chính xác, có căn cứ

Mục tiêu:

- Đưa ra câu trả lời cập nhật, chuyên sâu theo lĩnh vực và đáng tin cậy, đồng thời giảm hallucination.

2. RAG PIPELINE TỔNG QUAN



3. INGESTION LAYER (NGUỒN DỮ LIỆU)

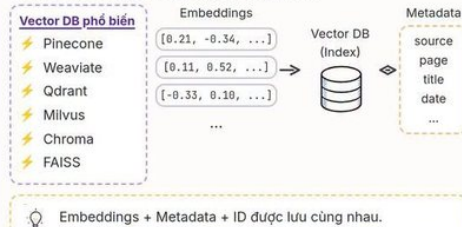


4. PREPARATION LAYER



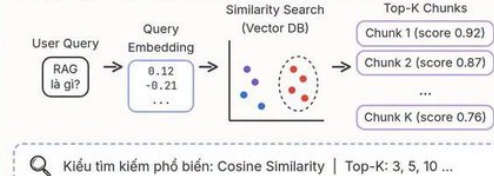
5. STORAGE LAYER (VECTOR DATABASE)

Lưu trữ embeddings để similarity search nhanh.



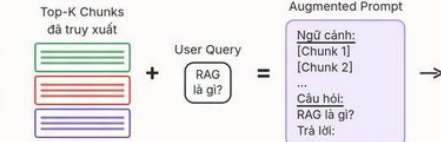
6. RETRIEVAL LAYER

Với một truy vấn của người dùng, tìm top-k chunk liên quan nhất.



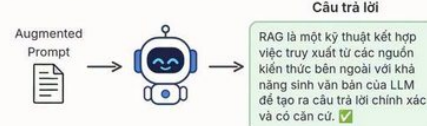
7. AUGMENTATION LAYER

Kết hợp ngữ cảnh truy xuất được với truy vấn của người dùng.

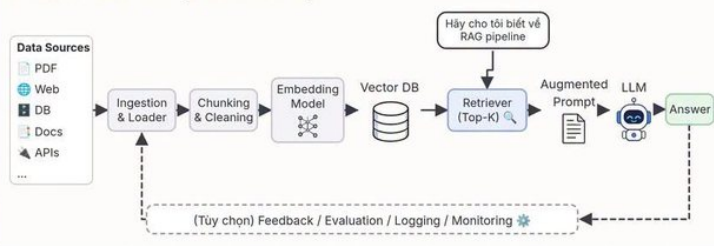


8. GENERATION LAYER (LLM)

LLM tạo câu trả lời dựa trên augmented prompt.



9. KIẾN TRÚC RAG (END-TO-END)



10. LỢI ÍCH CHÍNH CỦA RAG

- ✓ Dùng dữ liệu của chính bạn (riêng tư & chuyên ngành)
- ✓ Thông tin cập nhật
- ✓ Giảm hallucination
- ✓ Minh bạch & truy vết được (biết rõ nguồn)
- ✓ Tốt hơn cho doanh nghiệp & use case thực tế



CHƯƠNG 12 · GIẢI THÍCH CHI TIẾT

Pipeline & Kiến trúc RAG

1. Sáu lớp — mỗi lớp một trách nhiệm

Lớp	Đầu vào → đầu ra	Quyết định quan trọng
Ingestion	Nguồn → văn bản thô	Loader nào, lịch đồng bộ
Preparation	Văn bản thô → chunk sạch + vector	Chiến lược chunk, model embedding
Storage	Vector + metadata → chỉ mục	Vector DB, schema metadata
Retrieval	Câu hỏi → top-K chunk	Dense/sparse/hybrid, rerank, filter
Augmentation	Chunk + câu hỏi → prompt	Template, thứ tự chunk, ngân sách token
Generation	Prompt → câu trả lời	Model, temperature, trích dẫn

Tách lớp rõ ràng giúp thay thế từng phần độc lập (đối vector DB không ảnh hưởng prompt) và đo lường từng phần (Chương 14, "Component level").

2. Pipeline đầy đủ bằng Python (khung xương)

```
def ingest(paths):
    docs = [load(p) for p in paths]          # Ingestion
    chunks, metas = split_and_clean(docs)    # Preparation
    vecs = embedder.encode(chunks, normalize_embeddings=True)
    store.add(chunks, vecs, metas)           # Storage

def answer(question, user):
    qv = embedder.encode([question], normalize_embeddings=True)
    candids = store.search(qv, k=20,
                           filter={"doi_tuong": user.he_dao_tao}) # Retrieval
    top = rerank(question, candids)[:4]
    if not top or top[0].score < 0.3:         # ngưỡng an toàn
        return "Chưa tìm thấy quy định liên quan."
    prompt = TEMPLATE.format(ctx=format_ctx(top),
                              q=question)    # Augmentation
    return llm.generate(prompt, temperature=0.1) # Generation
```

Ba chi tiết đáng chú ý: (1) **filter metadata** theo đối tượng người hỏi trước khi tìm; (2) **ngưỡng điểm** — nếu chunk tốt nhất vẫn kém liên quan thì từ chối thay vì để model bịa; (3) **temperature thấp** cho câu trả lời dựa trên sự kiện.

3. Lớp Augmentation — những điều ảnh không nói

- Mỗi chunk nên kèm nhãn nguồn ([1] Quy chế, Điều 12) để model trích dẫn được.
- Thứ tự: đặt chunk liên quan nhất ở **đầu hoặc cuối** (tránh "lost in the middle").
- Đừng nhồi quá nhiều: 3–6 chunk chất lượng tốt hơn 20 chunk lẫn lộn.
- Với hội thoại nhiều lượt, câu hỏi tiếp theo kiểu "còn hệ liên thông thì sao?" phải được **viết lại thành câu hỏi độc lập** (dựa trên lịch sử) trước khi retrieval — nếu không, vector của "còn hệ liên thông thì sao?" gần như vô nghĩa.

4. Vòng phản hồi (đường nét đứt trong ảnh)

Đường "Feedback / Evaluation / Logging / Monitoring" nối từ Answer về Ingestion là phần biến một bản demo thành một sản phẩm:

- **Log** mọi câu hỏi, chunk được truy xuất, điểm, prompt, câu trả lời, độ trễ.
- **Thu phản hồi** (nút like/dislike) → các câu bị dislike là dữ liệu đánh giá quý nhất.
- **Phát hiện lỗi hỏng tri thức**: câu hỏi thường xuyên bị trả lời "chưa tìm thấy" cho biết tài liệu nào cần bổ sung.
- **Cập nhật chỉ mục**: khi văn bản thay thế được ban hành, đánh dấu văn bản cũ hết hiệu lực trong metadata.

5. Kiến trúc triển khai thực tế

Một cấu hình phổ biến cho trường học/doanh nghiệp vừa: API backend (FastAPI) chứa logic RAG; vector DB (Qdrant/pgvector) chạy cùng máy chủ; LLM qua API hoặc tự host (Ollama/vLLM trên GPU); giao diện nhúng vào hệ thống sẵn có (cổng thông tin, LMS) qua iframe hoặc plugin; một job định kỳ đồng bộ tài liệu mới. Phân quyền dựa trên danh tính đăng nhập của người dùng, áp dụng ở bước filter metadata.

CHƯƠNG 13 · ẢNH GỐC

13. Các kỹ thuật RAG nâng cao



13. CÁC KỸ THUẬT RAG NÂNG CAO



Truy xuất thông minh hơn. Ngữ cảnh sâu hơn. Câu trả lời tốt hơn. 🤖

1. VÌ SAO CẦN ADVANCED RAG?

- Basic RAG hoạt động tốt, nhưng dữ liệu thực tế thì lộn xộn, thay đổi liên tục và rất lớn.
- Các kỹ thuật nâng cao cải thiện mức độ liên quan, hiệu quả, độ chính xác & độ bền vững (robustness).
- Giảm hallucination và xử lý các truy vấn phức tạp.

Mục tiêu:

- Có đúng ngữ cảnh, đúng thời điểm, theo đúng cách.

2. HẠN CHẾ CỦA BASIC RAG

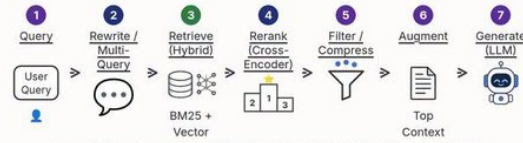
- Bỏ sót thông tin liên quan (recall kém)
- Trả về quá nhiều / quá ít ngữ cảnh
- Gặp khó với truy vấn multi-hop hoặc phức tạp
- Kiến thức trong LLM bị lỗi thời
- Tồn kém với context lớn



Advanced RAG giải quyết điều này!

- Recall tốt hơn với hybrid & multi-query
- Xếp hạng thông minh hơn với re-ranker
- Xử lý truy vấn phức tạp (decomposition)
- Cập nhật với iterative retrieval
- Chi phí thấp hơn nhờ compression & filtering

3. ADVANCED RAG PIPELINE (TỔNG QUAN)



Retrieval tốt hơn → Ngữ cảnh tốt hơn → Generation tốt hơn ❤️

4. QUERY REWRITING & MULTI-QUERY RETRIEVAL

Query Rewriting

Viết lại truy vấn của người dùng để cải thiện retrieval (rõ ràng hơn, từ đồng nghĩa, mở rộng).

Ví dụ:

Q: "Lợi ích của táo"

Viết lại: "Lợi ích sức khỏe của việc ăn táo"

Multi-Query Retrieval

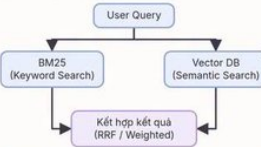
Tạo nhiều góc nhìn khác nhau cho truy vấn và truy xuất từ tất cả.

Các truy vấn ví dụ:

- Lợi ích của táo
- Giá trị dinh dưỡng của táo
- Táo có tốt cho sức khỏe không?
- Ưu điểm sức khỏe của táo

5. HYBRID RETRIEVAL

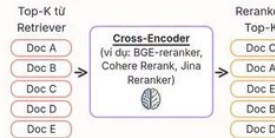
Kết hợp keyword (BM25) + semantic (vector) search để có recall tốt hơn.



Tốt cho cả thuật ngữ chính xác + ý nghĩa ngữ nghĩa.

6. RERANKING (CROSS-ENCODER)

Xếp hạng lại các tài liệu truy xuất được bằng cross-encoder model để tăng độ liên quan.



Cải thiện độ chính xác (precision) đáng kể.

7. CONTEXT COMPRESSION

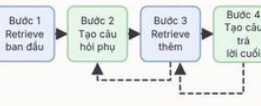
Giảm lượng token sử dụng mà vẫn giữ thông tin quan trọng.



Tiết kiệm token, giảm chi phí, sinh câu trả lời nhanh hơn.

8. ITERATIVE RETRIEVAL

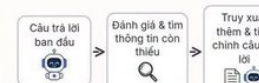
Retrieve → Tạo câu hỏi phụ → Retrieve thêm → Tinh chỉnh câu trả lời.



Rất tốt cho các câu hỏi phức tạp, multi-hop.

9. SELF-REFINE / SELF-CORRECTING RAG

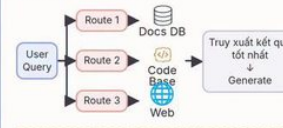
LLM kiểm tra câu trả lời, xác định chỗ còn thiếu, truy xuất thêm và cải thiện.



Cải thiện tính chính xác về dữ kiện & giảm hallucination.

10. ROUTING RAG (MULTI-SOURCE)

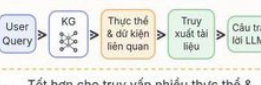
Định tuyến truy vấn đến nguồn dữ liệu hoặc index liên quan nhất.



Hữu ích khi dữ liệu nằm ở nhiều nguồn.

11. KNOWLEDGE GRAPH + RAG

Dùng Knowledge Graph để tăng cường retrieval bằng các mối quan hệ & thực thể.



Tốt hơn cho truy vấn nhiều thực thể & mang tính quan hệ.

12. ADAPTIVE CHUNKING

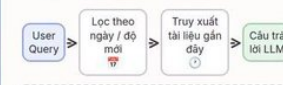
Tạo chunk một cách linh hoạt dựa trên loại nội dung.



Giữ được ngữ cảnh, cải thiện retrieval.

13. TIME-AWARE RAG

Ưu tiên truy xuất trong khoảng thời gian gần đây / còn hiệu lực.



Hữu ích cho tin tức, sự kiện, xu hướng.

CHƯƠNG 13 · GIẢI THÍCH CHI TIẾT

Các kỹ thuật RAG nâng cao

1. Bản đồ: mỗi kỹ thuật sửa một lỗi cụ thể

Lỗi quan sát được	Kỹ thuật
Câu hỏi ngắn, mơ hồ, dùng từ khác tài liệu	Query rewriting, multi-query, HyDE
Bỏ sót do khác từ khóa / khác cách diễn đạt	Hybrid retrieval
Có chunk đúng nhưng xếp hạng thấp	Reranking (cross-encoder)
Ngữ cảnh quá dài, tốn token, nhiễu	Context compression
Câu hỏi cần nhiều bước ("so sánh A và B")	Query decomposition, iterative retrieval
Câu trả lời không bám ngữ cảnh	Self-RAG / Corrective RAG / kiểm tra lại
Dữ liệu nằm ở nhiều nguồn khác bản chất	Routing
Câu hỏi về quan hệ giữa thực thể	Knowledge Graph / GraphRAG
Văn bản có nhiều phiên bản theo thời gian	Time-aware RAG (lọc theo ngày hiệu lực)

Đừng áp dụng tất cả. Hãy xem log lỗi (Chương 12) để biết hệ thống đang hỏng ở đâu, rồi chọn đúng kỹ thuật.

2. Biến đổi câu hỏi

- **Query rewriting:** LLM viết lại câu hỏi rõ ràng, đầy đủ hơn. "Lợi ích táo" → "Lợi ích sức khỏe của việc ăn táo". Với chatbot quy chế: "rớt môn có sao ko" → "Quy định khi sinh viên không đạt học phần".
- **Multi-query:** sinh 3–5 biến thể, retrieve cho từng biến thể, gộp kết quả (thường bằng RRF). Tăng recall, đổi lại thêm một lần gọi LLM và nhiều lần search.
- **HyDE** (Hypothetical Document Embeddings): bảo LLM viết một *câu trả lời giả định* rồi dùng embedding của câu trả lời đó để tìm. Lý do: câu trả lời giả định "trông giống" tài liệu thật hơn là câu hỏi. Nó có thể sai về sự kiện, nhưng chỉ dùng để tìm kiếm, không đưa cho người dùng.
- **Decomposition:** "Điều kiện tốt nghiệp của hệ chính quy và hệ liên thông khác nhau thế nào?" → hai câu hỏi con, retrieve riêng, tổng hợp.

3. Context compression

Ba cách trong ảnh: **extractive** (giữ lại câu liên quan), **abstractive** (tóm tắt), **LLM-based filter** (LLM chấm từng chunk có liên quan hay không). Với văn bản quy định, nên ưu tiên extractive vì tóm tắt có thể làm sai lệch câu chữ pháp lý.

4. Iterative, Self-RAG, FLARE, Corrective RAG

- **Iterative retrieval:** retrieve → sinh câu trả lời nháp → phát hiện còn thiếu gì → sinh câu hỏi phụ → retrieve thêm. Tốt cho câu hỏi multi-hop.
- **Self-RAG** (Asai và cộng sự, 2023): model được huấn luyện để tự quyết định khi nào cần truy xuất và tự đánh giá đoạn văn có liên quan / câu trả lời có được hỗ trợ không.
- **FLARE:** trong lúc sinh, nếu model sắp sinh các token có độ tự tin thấp thì dừng lại để truy xuất thêm.

- **Corrective RAG (CRAG)**: một bộ đánh giá chấm chất lượng tài liệu truy xuất; nếu kém thì viết lại câu hỏi hoặc chuyển sang nguồn khác (ví dụ web search).

5. Routing

Bộ định tuyến (một LLM nhỏ hoặc bộ phân loại) quyết định câu hỏi đi đâu: "lịch thi của tôi" → truy vấn API dữ liệu sinh viên; "quy định thi lại" → kho quy chế; "chào bạn" → trả lời trực tiếp không retrieve. Routing vừa tăng chính xác vừa giảm chi phí.

6. Knowledge Graph + RAG

Vector search giỏi tìm "đoạn văn giống câu hỏi" nhưng kém với câu hỏi về **quan hệ**: "Học phần nào là tiên quyết của học phần tiên quyết của Học máy?". Biểu diễn dữ liệu dưới dạng đồ thị (thực thể-quan hệ-thực thể) cho phép duyệt hai bước. GraphRAG (Microsoft, 2024) còn tóm tắt các cụm cộng đồng trong đồ thị để trả lời được câu hỏi toàn cục kiểu "các chủ đề chính trong kho tài liệu là gì?".

7. Adaptive chunking và time-aware RAG

- Adaptive chunking: chunk nhỏ cho đoạn chứa sự kiện, chunk lớn cho đoạn giải thích khái niệm, giữ nguyên bảng/danh sách thành một khối.
- Time-aware: lưu ngày hiệu lực trong metadata, lọc "chỉ văn bản còn hiệu lực tại ngày hỏi", hoặc cộng điểm cho tài liệu mới hơn. Rất quan trọng với quy chế được sửa đổi hằng năm — nếu không, chatbot có thể trích dẫn quy định đã bị thay thế.

8. Agentic RAG

Thay vì pipeline cố định, một agent (Chương 14) tự quyết định: có cần tìm không, tìm ở đâu, tìm lại với từ khóa khác, khi nào đủ thông tin để trả lời. Linh hoạt nhất nhưng khó dự đoán và tốn chi phí nhất — nên dùng khi pipeline cố định đã được đo là không đủ.

CHƯƠNG 14 · ẢNH GỐC

14. Ứng dụng LLM, Agents & Đánh giá



14. ỨNG DỤNG LLM, AGENTS & ĐÁNH GIÁ

RAG & LLMs
SERIES VIẾT TAY
v1.0

Use case thực tế. Agent thông minh. AI đáng tin cậy.

1. LLM ĐƯỢC DÙNG Ở ĐÂU?

LLM vận hành rất nhiều ứng dụng thực tế.



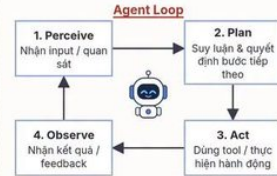
LLM có mục đích tổng quát - sự sáng tạo của bạn quyết định use case!

2. LLM AGENTS - LÀ GÌ & VÌ SAO?

Agent dùng LLM + tools + memory để tự động đạt được mục tiêu.

Vì sao cần Agent?

- ✓ Xử lý tác vụ phức tạp
- ✓ Lập kế hoạch, suy luận & hành động
- ✓ Dùng tools & dữ liệu bên ngoài
- ✓ Làm việc tự động
- ✓ Tốt hơn cho các tác vụ end-to-end



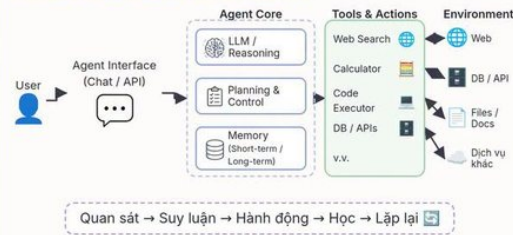
Think -> Plan -> Act -> Observe -> Lập lại cho đến khi đạt được mục tiêu

3. CÁC LOẠI LLM AGENT

Loại Agent	Mô tả	Ví dụ
Simple Reflex Agent	Hành động dựa trên input hiện tại.	FAQ bot
Tool-Using Agent	Dùng tools bên ngoài (search, calculator, API).	Code interpreter, web search agent
Planning Agent	Chia mục tiêu thành các bước và lập kế hoạch.	Agent lên kế hoạch du lịch
Memory-Based Agent	Dùng memory để duy trì ngữ cảnh qua các lượt.	Trợ lý cá nhân
Multi-Agent System	Nhiều agent phối hợp trong một tác vụ.	Nhóm agent nghiên cứu

★ Agents = LLMs + Reasoning + Tools + Memory

4. KIẾN TRÚC AGENT (THAM KHẢO)



Quan sát -> Suy luận -> Hành động -> Học -> Lập lại

5. CÁC AGENT FRAMEWORK PHỔ BIẾN

- LangChain**: Xây dựng ứng dụng LLM với chain, agent & memory.
- AutoGen**: Hội thoại & cộng tác giữa nhiều agent.
- CrewAI**: Các nhóm agent tự động dựa trên vai trò.
- LangGraph**: Xây dựng agent workflow có trạng thái (stateful), nhiều actor.
- LlamaIndex**: Kết nối dữ liệu, indexing & retrieval cho ứng dụng LLM.
- OpenAI Assistants API**: Xây dựng agent với tools, files & instructions.

Chọn framework phù hợp với use case & độ phức tạp của bạn.

6. USE CASE PHỔ BIẾN CỦA AGENT

- Trợ lý nghiên cứu**: Tìm kiếm -> Đọc -> Tóm tắt -> Báo cáo
- Agent hỗ trợ khách hàng**: Hiểu -> Tìm kiếm -> Trả lời -> Chuyển cấp
- Agent phân tích dữ liệu**: Truy vấn -> Phân tích -> Trực quan hóa -> Insight
- Agent lên kế hoạch du lịch**: Sở thích -> Lên kế hoạch -> Đặt chỗ -> Lịch trình
- Agent trợ lý lập trình**: Đọc code -> Gợi ý -> Test -> Cài thiện

Ví dụ: Luồng Research Agent

1. Người dùng đặt một câu hỏi nghiên cứu
2. Agent lên chiến lược tìm kiếm
3. Tìm kiếm & đọc nhiều nguồn
4. Tóm tắt các phát hiện
5. Viết một báo cáo có cấu trúc tốt kèm trích dẫn

7. EVALUATION - VÌ SAO?

Ta đánh giá LLM & agent để đảm bảo chúng chính xác, hữu ích, an toàn & đáng tin cậy.

- ✓ Đo lường chất lượng & hiệu năng
- ✓ Phát hiện hallucination & lỗi
- ✓ So sánh các model / prompt
- ✓ Cải thiện liên tục

Đánh giá những gì?

- ★ Tính đúng đắn
- ★ Mức độ liên quan
- ★ Tính đầy đủ
- ★ Tính mạch lạc
- ★ Faithfulness (không hallucination)
- ★ An toàn
- ★ Hiệu quả (Latency / Cost)

8. CÁC CẤP ĐỘ ĐÁNH GIÁ

- 1. Component Level**: Đánh giá prompt, retrieval, reranking, chunk, v.v.
- 2. System Level**: Đánh giá toàn bộ pipeline (ví dụ: hệ thống RAG, agent workflow).
- 3. User Experience Level**: Đánh giá mức độ hữu ích, sự hài lòng, khả năng sử dụng.

Đánh giá ở mọi cấp độ để có một hệ thống vững chắc!



9. CÁC METRIC ĐÁNH GIÁ PHỔ BIẾN

- ✓ **Accuracy / Correctness**: Câu trả lời có đúng về mặt dữ kiện không?
- 🎯 **Relevance**: Phản hồi liên quan đến mức nào?
- 🛡️ **Faithfulness**: Có dựa trên ngữ cảnh được cung cấp không?
- 🗨️ **Hallucination Rate**: Tần suất nó bịa ra thông tin?
- 📄 **Answer Completeness**: Có bao quát hết các ý chính không?
- 😊 **User Satisfaction**: Người dùng có thấy nó hữu ích không?

Metric tự động

- ★ BLEU / ROUGE
- ★ Exact Match (EM)
- ★ Semantic Similarity (Cosine / BERTScore)
- ★ Faithfulness Score
- ★ Groundedness Score

Human Evaluation

Tốt nhất để đánh giá chất lượng & tính hữu ích tổng thể.

CHƯƠNG 14 · GIẢI THÍCH CHI TIẾT

Ứng dụng LLM, Agents & Đánh giá

1. Từ chatbot đến agent

Chatbot RAG đi theo một đường cố định do lập trình viên viết. **Agent** để LLM tự quyết định bước tiếp theo trong một vòng lặp, dùng công cụ (tools) để tác động ra thế giới bên ngoài. Công thức trong ảnh: **Agent = LLM + Reasoning + Tools + Memory**.

2. Vòng lặp agent (ReAct)

Mẫu phổ biến nhất là ReAct (Reason + Act, Yao và cộng sự, 2022). Ví dụ một sinh viên hỏi "Tôi còn thiếu bao nhiêu tín chỉ để tốt nghiệp?":

- **Thought**: cần biết số tín chỉ đã tích lũy và yêu cầu của chương trình.
- **Action**: gọi tool `lay_bang_diem(ma_sv)`.
- **Observation**: đã tích lũy 112 tín chỉ.
- **Thought**: cần tổng tín chỉ của chương trình.
- **Action**: `tim_quy_che("tổng số tín chỉ chương trình CNTT")`.
- **Observation**: 150 tín chỉ.
- **Answer**: còn thiếu 38 tín chỉ (kèm nguồn).

3. Function calling — cách LLM "dùng tool"

LLM không tự chạy code. Bạn mô tả tool bằng JSON schema; model trả về một yêu cầu gọi tool có cấu trúc; **code của bạn** thực thi và gửi kết quả trở lại cho model.

```
tools = [{
  "name": "lay_bang_diem",
  "description": "Lấy bảng điểm và số tín chỉ tích lũy của sinh viên",
  "input_schema": {
    "type": "object",
    "properties": {"ma_sv": {"type": "string"}},
    "required": ["ma_sv"]
  }
}]
# Model trả về: {"type": "tool_use", "name": "lay_bang_diem",
#               "input": {"ma_sv": "..."}}
# -> code của bạn gọi API, trả kết quả lại cho model
```

Bảo mật: tham số định danh (mã sinh viên) nên lấy từ phiên đăng nhập phía server, **không** tin vào giá trị model đưa ra — nếu không, người dùng có thể dụ model tra điểm của người khác. Tiêu chuẩn mở **MCP** (Model Context Protocol) đang được dùng rộng rãi để kết nối LLM với tool/dữ liệu theo một giao thức thống nhất.

4. Memory

- **Ngắn hạn**: lịch sử hội thoại trong context window.
- **Dài hạn**: lưu thông tin ra ngoài (database, vector store) và truy xuất lại khi cần — về bản chất cũng là một dạng RAG.

5. Các loại agent và framework

Bảng loại agent trong ảnh sắp xếp từ đơn giản (reflex) đến phức tạp (multi-agent). Framework: LangChain (khối xây dựng chung), LangGraph (workflow có trạng thái, dạng đồ thị — kiểm soát tốt hơn), LlamaIndex (mạnh về indexing/RAG), AutoGen và CrewAI (nhiều agent cộng tác). Lời khuyên: hiểu nguyên lý bằng code thuần trước (một vòng while gọi LLM và tool), sau đó mới dùng framework — sẽ đỡ lỗi dễ hơn nhiều.

6. Đánh giá — phân quyết định chất lượng

Ba cấp độ trong ảnh tương ứng ba câu hỏi: từng thành phần có tốt không (component), toàn hệ thống có trả lời đúng không (system), người dùng có hài lòng không (UX). Với RAG, tách riêng đánh giá **retrieval** và **generation** là bắt buộc — nếu không, khi câu trả lời sai bạn không biết lỗi do tìm sai hay do model hiểu sai.

7. Metric cho retrieval — tính tay

Giả sử với một câu hỏi có 2 chunk đúng (A, C), hệ thống trả top-5: [B, A, D, C, E].

- **Recall@5** = số chunk đúng tìm được / tổng chunk đúng = $2/2 = 1,0$.
- **Precision@5** = $2/5 = 0,4$.
- **MRR**: chunk đúng đầu tiên ở hạng 2 $\rightarrow 1/2 = 0,5$ (lấy trung bình trên nhiều câu hỏi).
- **Hit@1** = 0 (hạng 1 là B, sai).

8. Metric cho generation

Metric	Câu hỏi nó trả lời	Đo thế nào
Faithfulness / Groundedness	Mọi khẳng định trong câu trả lời có được ngữ cảnh hỗ trợ?	Tách thành các khẳng định, LLM kiểm tra từng cái
Answer relevancy	Câu trả lời có đúng trọng tâm câu hỏi?	LLM-as-judge hoặc so embedding
Correctness	Có đúng so với đáp án chuẩn?	So với ground truth
Context precision/recall	Ngữ cảnh truy xuất có đủ và gọn?	Cần đáp án chuẩn

Thư viện như RAGAS, DeepEval, TruLens tự động hóa các metric này bằng **LLM-as-a-judge**. BLEU/ROUGE (đếm n-gram trùng) ít phù hợp cho hỏi đáp mở vì một câu trả lời đúng có thể dùng từ hoàn toàn khác đáp án mẫu.

Thực hành tối thiểu: xây một **bộ đánh giá vàng** 50-100 câu hỏi thật, mỗi câu có đáp án chuẩn và chunk nguồn đúng. Mỗi lần đổi chunk size, model embedding, prompt hay reranker, chạy lại toàn bộ và so sánh số liệu. Không có bộ này, mọi cải tiến chỉ là cảm tính.

CHƯƠNG 15 · ẢNH GỐC

15. Phỏng vấn RAG & LLM + Kiến thức thực tế



15. PHÒNG VẤN RAG & LLM + KIẾN THỨC THỰC TẾ

Hiểu sâu. Giải thích rõ. Xây dựng tự tin. 🎯

RAG & LLMs
SERIES VIẾT TAY
v1.0



1. CÂU HỎI PHÒNG VẤN RAG HÀNG ĐẦU

- RAG là gì và vì sao nó quan trọng?
- RAG giảm hallucination như thế nào?
- Giải thích RAG pipeline.
- Các thành phần chính của RAG là gì?
- Embeddings hoạt động thế nào trong RAG?
- Chunking là gì? Bạn quyết định chunk size như thế nào?
- Bạn chọn vector database như thế nào?
- Reranking là gì? Vì sao cần nó?
- Bạn xử lý tài liệu lớn như thế nào?
- Bạn đánh giá một hệ thống RAG như thế nào?
- Những thách thức trong RAG là gì?
- Bạn sẽ cải thiện RAG về độ chính xác / độ trễ như thế nào?

2. CÂU HỎI PHÒNG VẤN LLM

- Sự khác biệt giữa pretraining, fine-tuning, instruction tuning & RLHF là gì?
- Cơ chế attention hoạt động thế nào?
- Context window là gì?
- Temperature, top-p, top-k là gì?
- Giải thích beam search vs greedy decoding.
- Transformers là gì?
- Tokenization là gì?
- Bạn xử lý bias & safety trong LLM như thế nào?
- Bạn giảm chi phí khi dùng LLM như thế nào?
- Giải thích function calling / tools trong LLM.
- Sự khác biệt giữa RAG và Fine-tuning là gì?

3. RAG PIPELINE - TÓM TẮT NHANH







Tim các chunk liên quan → Chọn top-K liên quan nhất → Thêm ngữ cảnh vào prompt → LLM tạo câu trả lời

Thành phần chính

- ✓ Ingestion, Chunking, Embedding, Vector DB
- ✓ Retrieval, Reranking, Prompting, Generation
- ✓ Logging, Monitoring, Evaluation

4. USE CASE RAG THỰC TẾ

Use Case	Tài liệu công ty, hướng dẫn sản phẩm → câu trả lời chính xác	Tài liệu nội bộ, wiki, SOP → trợ lý cho nhân viên	Bài báo khoa học, bài viết → tóm tắt, phân tích, trích dẫn	Hợp đồng, án lệ → tìm kiếm điều khoản, Q&A	Hướng dẫn y khoa, báo cáo → hỗ trợ chuyên môn	Tài liệu sản phẩm, đánh giá → gợi ý thống minh	Báo cáo, hồ sơ tài chính → insight, phân tích rủi ro	Ghi chú, sách → giải đáp thắc mắc, gia sư
FAQs / Helpdesk								
Knowledge Base								
Research Assistant								
Legal / Contracts								
Healthcare								
E-commerce								
Finance								
Education								

RAG kết hợp dữ liệu riêng của bạn + khả năng hiểu của LLM.

5. CÁC QUYẾT ĐỊNH THIẾT KẾ QUAN TRỌNG

Quyết định	Điều cần cân nhắc
Chunk Size	Nhỏ hơn = chính xác, Lớn hơn = nhiều ngữ cảnh (thường dùng 200-500 token)
Chunk Overlap	Giúp duy trì ngữ cảnh giữa các chunk (overlap 10-20%)
Embedding Model	Chất lượng vs Chi phí vs Tốc độ (ví dụ: text-embedding-3-large)
Vector DB	Quy mô, filtering, hybrid search, chi phí, managed vs self-hosted
Reranker	Cross-encoder cho độ chính xác, nhưng tăng latency & chi phí
Top-K	K cao hơn = recall tốt hơn, nhưng chi phí & latency cao hơn
Prompt Design	Chỉ dẫn rõ ràng, trích dẫn nguồn, giới hạn phạm vi
Generation Params	Temperature, top-p, max tokens ảnh hưởng đến phong cách câu trả lời & chi phí

6. THÁCH THỨC RAG & CÁCH GIẢI QUYẾT

Thách thức	Cách khắc phục
Truy xuất không liên quan	Embeddings tốt hơn, metadata filter, hybrid search, chunking tốt
Hallucinations	Dùng ngữ cảnh chất lượng cao, grounding, trích dẫn, temperature thấp
Context / Memory lớn	Tóm tắt, map-reduce RAG, hierarchical retrieval
Latency	Caching, chỉ rerank top-N, async pipeline, model nhỏ hơn
Chi phí	Model nhỏ hơn, retrieval có chọn lọc, nén ngữ cảnh, batching
Dữ liệu cũ	Ingestion dữ liệu mới, versioning, lọc theo thời gian
Bảo mật / Quyền riêng tư	Kiểm soát truy cập, che dữ liệu (data masking), loại bỏ PII, mã hóa

7. CHIẾN LƯỢC TÌM KIẾM TRONG RAG

- Dense Search (Semantic)**
Tốt nhất cho truy xuất dựa trên ý nghĩa
- Sparse Search (BM25)**
Tốt nhất cho từ khóa & khớp chính xác
- Hybrid Search (Dense + Sparse)**
Kết hợp recall của BM25 + sức mạnh ngữ nghĩa
- Multi-Query Retrieval**
Tạo nhiều truy vấn từ truy vấn của người dùng
- Self-Query Retrieval**
LLM chuyển query → filter → search

Hybrid + Rerank = Cân bằng tốt nhất giữa Recall & Precision

8. RERANKING - VÌ SAO & THẾ NÀO

Trước Reranking (theo similarity)	Cross-Encoder Reranker	Sau Reranking (theo relevance)
1. Doc A (score: 0.78)	→	1. Doc C (relevance: 0.92)
2. Doc B (score: 0.74)		2. Doc A (relevance: 0.89)
3. Doc C (score: 0.65)		3. Doc E (relevance: 0.76)
4. Doc D (score: 0.61)		4. Doc B (relevance: 0.63)
5. Doc E (score: 0.58)		5. Doc D (relevance: 0.45)

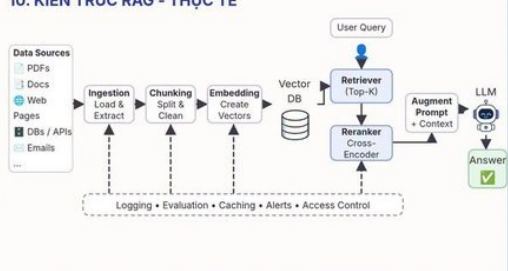
Reranking cải thiện độ chính xác nhờ hiểu sâu mối quan hệ giữa query và tài liệu.

9. METRIC ĐÁNH GIÁ CHO RAG

- Retrieval Metrics**
Recall@K, Precision@K, MRR, nDCG
- Generation Metrics**
Faithfulness, Correctness, Answer Relevancy
- End-to-End Metrics**
Answer Accuracy, Helpfulness, Completeness
- Human Evaluation**
User Satisfaction, Safety, Usability

Đo lường cả chất lượng retrieval + chất lượng câu trả lời.

10. KIẾN TRÚC RAG - THỰC TẾ



The diagram shows the flow from Data Sources (PDFs, Docs, Web Pages, DBs/APIs, Emails) through Ingestion (Load & Extract), Chunking (Split & Clean), Embedding (Create Vectors) into a Vector DB. A User Query is processed by a Retriever (Top-K) and a Reranker (Cross-Encoder) to produce an Answer. The process is monitored by Logging, Evaluation, Caching, Alerts, and Access Control.

11. LLM AGENTS - THỰC TẾ

- Customer Support Agent**
Trả lời FAQ, chuyển cấp, tạo ticket
- Data Analyst Agent**
Truy vấn dữ liệu, tạo biểu đồ, insight
- Travel Planner Agent**
Lên kế hoạch chuyến đi, đặt chỗ, gợi ý lịch trình
- Code Assistant Agent**
Viết code, debug, giải thích, refactor
- Research Agent**
Tìm kiếm, đọc, tóm tắt, báo cáo

Agent Loop

```

        graph TD
            Perceive --> Plan
            Plan --> Act[Act Tool Use]
            Act --> Observe
            Observe --> Perceive
            
```

Agent dùng suy luận + tools + memory để hoàn thành công việc.

CHƯƠNG 15 · GIẢI THÍCH CHI TIẾT

Phỏng vấn RAG & LLM + Kiến thức thực tế

1. Gợi ý trả lời các câu hỏi phỏng vấn RAG

RAG giảm hallucination như thế nào? Cung cấp bằng chứng trong ngữ cảnh, kết hợp chỉ dẫn "chỉ trả lời từ tài liệu", cho phép từ chối khi không có thông tin, và trích dẫn để kiểm chứng. RAG *giảm* chứ không *loại bỏ*: model vẫn có thể hiểu sai hoặc kết hợp sai ngữ cảnh.

Bạn quyết định chunk size như thế nào? Theo bản chất tài liệu (văn bản quy định → theo Điều/Khoản), theo giới hạn của embedding model, và bằng thực nghiệm: thử 256/512/1024 token trên bộ đánh giá, chọn theo Recall@k và chất lượng câu trả lời.

Chọn vector DB như thế nào? Quy mô, nhu cầu lọc metadata, hybrid search, managed hay tự host, hạ tầng sẵn có (đã có Postgres → pgvector), chi phí vận hành.

Vì sao cần reranking? Bi-encoder nén tài liệu thành vector trước khi biết câu hỏi nên mất chi tiết; cross-encoder đọc cặp câu hỏi-tài liệu cùng lúc nên chính xác hơn, nhưng chậm — vì vậy chỉ áp dụng trên top-N nhỏ.

Xử lý tài liệu lớn thế nào? Chunk có cấu trúc, phân cấp (tóm tắt mức tài liệu/mục → chi tiết), parent-document retrieval, map-reduce khi cần tổng hợp toàn bộ.

Đánh giá hệ RAG thế nào? Tách retrieval (Recall@k, MRR) và generation (faithfulness, relevancy, correctness), dùng bộ đánh giá vàng + LLM-as-judge + phản hồi người dùng.

Cải thiện độ chính xác / độ trễ? Chính xác: dữ liệu sạch, chunk tốt, hybrid, rerank, prompt có trích dẫn. Độ trễ: cache câu hỏi lặp lại và embedding, rerank ít ứng viên hơn, model nhỏ hơn cho bước phụ, streaming câu trả lời, chạy các bước độc lập song song.

2. Gợi ý trả lời các câu hỏi về LLM

Temperature, top-k, top-p là gì? Temperature chia logits trước softmax:

$$P_i = e^{z_i/T} / \sum_j e^{z_j/T}$$

Với logits [2; 1; 0]: $T = 1 \rightarrow [0,67; 0,24; 0,09]$; $T = 0,5 \rightarrow [0,87; 0,12; 0,02]$ (tập trung, ổn định); $T = 2 \rightarrow [0,51; 0,31; 0,19]$ (đa dạng, ngẫu nhiên). **Top-k**: chỉ lấy mẫu trong k token xác suất cao nhất. **Top-p** (nucleus): lấy tập nhỏ nhất các token có tổng xác suất $\geq p$ (ví dụ $p = 0,9$) — kích thước tập tự co giãn theo độ tự tin của model. Với RAG trả lời sự kiện: temperature thấp (0-0,3).

Beam search vs greedy? Greedy chọn token xác suất cao nhất ở mỗi bước — nhanh nhưng có thể bỏ lỡ câu tổng thể tốt hơn. Beam search giữ B ứng viên tốt nhất song song ở mỗi bước, chọn chuỗi có xác suất tổng cao nhất — tốt cho dịch máy; với chat thường dùng sampling vì beam search hay sinh văn bản nhặt và lặp.

Giảm chi phí khi dùng LLM? Chọn model nhỏ nhất đạt yêu cầu; routing (câu dễ → model rẻ); prompt caching; rút gọn ngữ cảnh; batching; giới hạn max_tokens; cache câu trả lời cho câu hỏi phổ biến.

RAG khác fine-tuning thế nào? RAG thay đổi *những gì model đọc* lúc hỏi (kiến thức, cập nhật, trích dẫn); fine-tuning thay đổi *bản thân model* (hành vi, văn phong, định dạng). Xem bảng ở Chương 6.

Xử lý bias & safety? Lọc đầu vào/đầu ra (moderation), system prompt có nguyên tắc rõ, phân quyền dữ liệu, phòng chống prompt injection (tách chỉ dẫn và dữ liệu, không cấp tool nguy hiểm), đánh giá trên bộ test đối kháng, có người giám sát với quyết định quan trọng.

3. Các quyết định thiết kế — bảng giá trị khởi điểm

Tham số	Giá trị khởi điểm hợp lý
Chunk size	300-600 token (văn bản quy định: theo Điều)
Overlap	10-20%
Top-N retrieval	20-50
Top-K sau rerank	3-5
Temperature	0-0,3 cho hỏi đáp sự kiện
Ngưỡng từ chối	Hiệu chỉnh trên bộ đánh giá

4. Bảng thách thức & cách giải quyết — đọc theo chẩn đoán

Khi chatbot trả lời sai, hỏi theo thứ tự: (1) Chunk đúng có **tồn tại** trong DB không? (lỗi loading/chunking) → (2) Có nằm trong **top-N** không? (lỗi embedding/hybrid) → (3) Có lọt vào **top-K** sau rerank không? (lỗi reranker) → (4) Có trong prompt mà model vẫn trả lời sai? (lỗi prompt/model). Quy trình chẩn đoán này là câu trả lời "ăn điểm" cho câu hỏi system design.

5. Câu hỏi system design mẫu

"Thiết kế chatbot hỏi đáp quy chế cho 20.000 sinh viên." Một khung trả lời tốt:

- Yêu cầu: chính xác, trích dẫn, tiếng Việt, phân quyền theo hệ đào tạo, độ trễ < 5 giây, chi phí kiểm soát được.
- Dữ liệu: văn bản quy chế (PDF/DOCX) chunk theo Điều/Khoản, metadata gồm văn bản, Điều, ngày hiệu lực, đối tượng.
- Retrieval: hybrid (BM25 + embedding đa ngôn ngữ) → rerank → top-4, lọc theo đối tượng và hiệu lực.
- Generation: prompt có trích dẫn và lối thoát "không tìm thấy", temperature thấp.
- Dữ liệu cá nhân (điểm, tín chỉ): qua tool gọi API với định danh lấy từ phiên đăng nhập, không đưa vào vector DB.
- Vận hành: log, phản hồi người dùng, bộ đánh giá vàng, đồng bộ khi có văn bản mới, giới hạn tần suất truy vấn.

6. Lời kết

Bộ cẩm nang này đi từ "LLM dự đoán token tiếp theo" đến "agent tự lập kế hoạch". Sợi chỉ xuyên suốt: **LLM là bộ máy sinh văn bản mạnh nhưng không đáng tin tuyệt đối; giá trị thật nằm ở cách ta cung cấp đúng dữ liệu, đúng lúc, và đo lường cẩn thận kết quả.** Cách học hiệu quả nhất là tự xây một hệ RAG nhỏ từ đầu, đo nó bằng bộ đánh giá của chính mình, rồi cải tiến từng bước theo số liệu.

PHỤ LỤC

Phụ lục: Bảng thuật ngữ

Thuật ngữ	Giải thích ngắn
Token	Đơn vị văn bản model xử lý (từ, mảnh từ, ký tự)
Embedding	Vector số biểu diễn ý nghĩa của token hoặc đoạn văn
Context window	Số token tối đa model xử lý trong một lần gọi
Attention	Cơ chế cho mỗi token tổng hợp thông tin từ các token khác theo trọng số
Logits	Điểm thô trước softmax cho mỗi token trong từ vựng
Hallucination	Model sinh nội dung trôi chảy nhưng sai sự thật
Pretraining	Huấn luyện trên dữ liệu khổng lồ không nhãn (next-token prediction)
SFT	Supervised Fine-Tuning trên cặp chỉ dẫn-phản hồi
RLHF	Học tăng cường từ phản hồi của con người
DPO	Tối ưu trực tiếp theo sở thích, không cần reward model
LoRA / QLoRA	Fine-tune bằng ma trận hạng thấp / kết hợp lượng tử hóa 4-bit
Chunk	Mảnh tài liệu nhỏ được index và truy xuất
Overlap	Phần chồng lấn giữa hai chunk liền kề
Bi-encoder	Model mã hóa query và tài liệu riêng rẽ (dùng cho retrieval)
Cross-encoder	Model chấm điểm cặp query-tài liệu cùng lúc (dùng cho rerank)
BM25	Thuật toán xếp hạng theo từ khóa (sparse retrieval)
Hybrid search	Kết hợp BM25 và vector search
RRF	Reciprocal Rank Fusion — gộp nhiều bảng xếp hạng theo thứ hạng
MMR	Maximal Marginal Relevance — cân bằng liên quan và đa dạng
ANN / HNSW	Tìm láng giềng gần đúng / chỉ mục đồ thị nhiều tầng
HyDE	Tìm kiếm bằng embedding của câu trả lời giả định
Recall@k	Tỷ lệ tài liệu đúng nằm trong top-k
MRR	Trung bình nghịch đảo hạng của kết quả đúng đầu tiên
Faithfulness	Mức độ câu trả lời được ngữ cảnh hỗ trợ
Function calling	LLM trả về yêu cầu gọi tool có cấu trúc
ReAct	Mẫu agent xen kẽ suy luận và hành động
MCP	Model Context Protocol — chuẩn kết nối LLM với tool/dữ liệu
Prompt injection	Tấn công chen chỉ dẫn độc hại vào đầu vào hoặc tài liệu
KV cache	Bộ nhớ đệm K, V của token cũ để tăng tốc sinh văn bản

Lưu ý về trích dẫn: các bài báo và công cụ được nhắc tên trong sách (Attention Is All You Need, Lost in the Middle, Self-RAG, ReAct, LoRA, DPO, GraphRAG, RAGAS...) được nêu theo hiểu biết của người biên soạn, không tra cứu trực tiếp khi viết; tên model và số liệu thay đổi nhanh — hãy kiểm tra lại với nguồn gốc trước khi trích dẫn chính thức.